

گزارش فنی، ممیزی و راهنمای توسعه

سامانه پایش و کنترل اتاق آزمون

مبتنی بر STM32F107VCT6

نسخه سند: ۰.۱

تاریخ ممیزی: شهریور ۱۴۰۵

راهنمای استفاده از سند

این گزارش براساس خواندن همه فایل‌های سورس، تنظیمات STM32CubeMX، پروژه Keil MDK و خروجی ساخت موجود تدوین شده است. ادعاهای عملکردی که تنها با اتصال برد قابل اثبات‌اند، با عنوان «نیازمند آزمون سخت‌افزاری» مشخص شده‌اند. فایل تنظیمات ظاهری گزارش docs/report-config.tex است و هر فصل در پوشه docs/chapters به‌صورت مستقل نگهداری می‌شود.

فهرست مطالب

۱	دامنه، هدف و نمای کلی سامانه	۱
۱	۱.۱ کاربرد	۱
۱	۲.۱ حدود نسخه حاضر	۱
۱	۳.۱ نمای جریان داده	۱
۲	۴.۱ واحدهای مهم	۲
۳	۲ سخت افزار و تنظیمات مولد کد	۳
۳	۱.۲ پردازنده و کلاک	۳
۳	۲.۲ پیرامونی های فعال	۳
۴	۳.۲ پایه های کاربردی قطعی	۴
۵	۴.۲ ایمنی تولید مجدد	۵
۶	۳ معماری نرم افزار و مالکیت فایل ها	۶
۶	۱.۳ لایه ها	۶
۶	۲.۳ قرارداد با فایل های تولیدی	۶
۶	۳.۳ زمان بندی تعاونی	۶
۷	۴.۳ هم رندی	۷
۷	۵.۳ حافظه پایدار	۷
۸	۴ درایورها، ماژول ها و وضعیت استفاده	۸

۸	درایور دمای دیجیتال	۱.۴
۸	اندازه‌گیری انرژی	۲.۴
۹	رطوبت آنالوگ	۳.۴
۹	حسگر رطوبت دیجیتال	۴.۴
۹	فیلتر میانگین متحرک	۵.۴
۹	درایورهای HAL	۶.۴
۱۰	۵ رفتار زمان اجرا و کنترل محیط	
۱۰	راه‌اندازی	۱.۵
۱۰	کنترل رطوبت	۲.۵
۱۰	کنترل سرمایش	۳.۵
۱۱	کنترل گرمایش	۴.۵
۱۱	حالت‌ها و ایمنی	۵.۵
۱۲	۶ پروتکل سریال رایانه	
۱۲	لایه انتقال	۱.۶
۱۲	فرمان‌ها	۲.۶
۱۳	قاب داده ۲۰۹ بایتی	۳.۶
۱۵	۷ نتیجه ممیزی و تغییرات انجام‌شده	
۱۵	نقاط قوت نسخه اولیه	۱.۷
۱۵	اشکال‌های بحرانی اصلاح‌شده	۲.۷
۱۶	موارد حفظ‌شده	۳.۷
۱۶	ریسک‌های باز	۴.۷
۱۶	چرا بعضی فایل‌ها حذف نشدند	۵.۷

۱۷	۸	ساخت، پروگرام و تولید مجدد
۱۷	۱.۸	ساخت با Keil
۱۷	۲.۸	ساخت با PlatformIO
۱۸	۳.۸	تولید مجدد با CubeMX
۱۸	۴.۸	ساخت گزارش
۱۹	۹	تأیید، تحویل و نگهداری آینده
۱۹	۱.۹	آزمون رومیزی بدون بار قدرت
۱۹	۲.۹	آزمون پروتکل
۱۹	۳.۹	آزمون کنترل
۲۰	۴.۹	معیار پذیرش نسخه
۲۰	۵.۹	اولویتهای توسعه بعدی
۲۰	۶.۹	قاعده نگهداری سند
۲۱	۱۰	مرجع کامل توابع لایه کاربرد
۲۱	۱.۱۰	ساختار ورودی راهاندازی
۲۱	۲.۱۰	توابع عمومی چرخه اجرا
۲۱	۱.۲.۱۰	تابع App_Init
۲۲	۲.۲.۱۰	تابع App_Process
۲۲	۳.۱۰	توابع داخلی تبدیل و ایمنی
۲۲	۱.۳.۱۰	تابع outputs_safe
۲۲	۲.۳.۱۰	تابع clamp_u16_from_float
۲۲	۳.۳.۱۰	تابع put_u16_le
۲۳	۴.۱۰	اندازه‌گیری و محاسبه
۲۳	۱.۴.۱۰	تابع sample_electrical

۲۳		تابع read_temperature_deci_c	۲.۴.۱۰
۲۳		تابع sample_temperatures	۳.۴.۱۰
۲۳		کنترل خروجی‌ها	۵.۱۰
۲۴		تابع apply_automatic_control	۱.۵.۱۰
۲۴		ساخت و ارسال داده	۶.۱۰
۲۴		تابع transmit_telemetry	۱.۶.۱۰
۲۴		دریافت و تفسیر فرمان	۷.۱۰
۲۴		تابع command_is	۱.۷.۱۰
۲۴		تابع clear_command	۲.۷.۱۰
۲۵		تابع process_command	۳.۷.۱۰
۲۵		توابع ورودی از وقفه	۸.۱۰
۲۵		تابع App_OnAdcConversionComplete	۱.۸.۱۰
۲۵		تابع App_OnTimerElapsed	۲.۸.۱۰
۲۵		تابع App_OnUartRxComplete	۳.۸.۱۰
۲۵		تابع App_OnUartError	۴.۸.۱۰
۲۶		توابع HAL بازنویسی شده	۵.۸.۱۰
۲۶		داده‌های سراسری مهم	۹.۱۰

۱۱ مرجع کامل توابع درایورها ۲۷

۲۷		ذخیره شناسه حسگر	۱.۱۱
۲۷		تابع crc8_maxim	۱.۱.۱۱
۲۷		تابع crc32_record	۲.۱.۱۱
۲۷		تابع SensorAddressStore_IsValidRom	۳.۱.۱۱
۲۸		تابع table_is_valid	۴.۱.۱۱
۲۸		تابع SensorAddressStore_Load	۵.۱.۱۱

۲۸		SensorAddressStore_Save	تابع	۶.۱.۱۱
۲۸		ADE7880	درایور	۲.۱۱
۲۸		ADE7880_ReadRegister	تابع	۱.۲.۱۱
۲۹		ADE7880_WriteRegister	تابع	۲.۲.۱۱
۲۹		ADE7880_ConfigSPI	تابع	۳.۲.۱۱
۲۹		ADE7880_Run	تابع	۴.۲.۱۱
۲۹		ADE7880_ReadVRMS	تابع	۵.۲.۱۱
۲۹		ADE7880_ReadIRMS	تابع	۶.۲.۱۱
۲۹		ADE7880_ReadPowerz	تابع	۷.۲.۱۱
۳۰		ADE7880_ReadFPower	تابع	۸.۲.۱۱
۳۰		ADE7880_Reset	تابع	۹.۲.۱۱
۳۰		DS18B20	درایور دوگانه	۳.۱۱
۳۰		DS18B20_Init	تابع	۱.۳.۱۱
۳۰		توابع زمان و شمار		۲.۳.۱۱
۳۰		توابع نوشتن		۳.۳.۱۱
۳۱		توابع تغییر جهت پایه		۴.۳.۱۱
۳۱		توابع خواندن		۵.۳.۱۱
۳۱		توابع انتظار و بازنشانی		۶.۳.۱۱
۳۱		توابع انتخاب شناسه		۷.۳.۱۱
۳۱		توابع خواندن دما		۸.۳.۱۱
۳۱		AM2315	درایور	۴.۱۱
۳۲		am2315_crc16	تابع	۱.۴.۱۱
۳۲		TEMP_HUM	تابع	۲.۴.۱۱
۳۲		MAF	فیلتر	۵.۱۱

۳۲	 تابع MAF_Init	۱.۵.۱۱
۳۲	 تابع MAF_Update	۲.۵.۱۱
۳۲	 تابع Average	۳.۵.۱۱
۳۳	۱۲ شرح جزئی بازآرایی و مرز تغییرات	
۳۳	 وضعیت پیش از بازآرایی	۱.۱۲
۳۳	 انتقال مالکیت	۲.۱۲
۳۳	 تبدیل وقفه به کار معوق	۳.۱۲
۳۴	 فشرده سازی قاب مخابراتی	۴.۱۲
۳۴	 بازطراحی دریافت سریال	۵.۱۲
۳۴	 بازطراحی حافظه شناسه ها	۶.۱۲
۳۴	 اصلاح رفتار کنترل	۷.۱۲
۳۵	 مرزهای حفاظت شده CubeMX	۸.۱۲
۳۵	 مواردی که عمداً تغییر نکردند	۹.۱۲
۳۵	 موارد باقیمانده برای مرحله بعد	۱۰.۱۲
۳۶	۱۳ مرجع توابع تولیدشده و راه اندازی سخت افزار	
۳۶	 فایل اصلی	۱.۱۳
۳۶	 تابع main	۱.۱.۱۳
۳۶	 تابع SystemClock_Config	۲.۱.۱۳
۳۷	 تابع MX_ADC1_Init	۳.۱.۱۳
۳۷	 تابع MX_I2C1_Init	۴.۱.۱۳
۳۷	 تابع MX_SPI2_Init	۵.۱.۱۳
۳۷	 توابع MX_TIM1_Init، MX_TIM2_Init و MX_TIM3_Init	۶.۱.۱۳
۳۷	 تابع MX_USART2_UART_Init	۷.۱.۱۳

۳۷		تابع MX_DMA_Init	۸.۱.۱۳
۳۸		تابع MX_GPIO_Init	۹.۱.۱۳
۳۸		تابع Error_Handler	۱۰.۱.۱۳
۳۸		توابع پشتیبانی پیرامونی	۲.۱۳
۳۸		تابع HAL_MspInit	۱.۲.۱۳
۳۸		توابع HAL_ADC_MspInit و HAL_ADC_MspDeInit	۲.۲.۱۳
۳۸		توابع HAL_I2C_MspInit و HAL_I2C_MspDeInit	۳.۲.۱۳
۳۹		توابع HAL_SPI_MspInit و HAL_SPI_MspDeInit	۴.۲.۱۳
۳۹		توابع پشتیبانی تایمرها	۵.۲.۱۳
۳۹		توابع HAL_UART_MspInit و HAL_UART_MspDeInit	۶.۲.۱۳
۳۹		هندلرهای استثنا و وقفه	۳.۱۳
۳۹		فایل سامانه و راهانداز	۴.۱۳
۴۰	۱۴ نشانگر زنده‌بودن و آزمون دو چراغ		
۴۰		هدف آزمون	۱.۱۴
۴۰		اتصال الکتریکی و قطب فعال	۲.۱۴
۴۱		تنظیمات مولد کد	۳.۱۴
۴۱		فایل‌ها و رابط عمومی	۴.۱۴
۴۱		تابع DiagnosticLed_Init	۱.۴.۱۴
۴۱		تابع DiagnosticLed_SetMode	۲.۴.۱۴
۴۱		تابع DiagnosticLed_Process	۳.۴.۱۴
۴۲		تابع DiagnosticLed_GetMode	۴.۴.۱۴
۴۲		الگوهای قابل انتخاب	۵.۱۴
۴۲		حالت DIAGNOSTIC_LED_MODE_OFF	۱.۵.۱۴
۴۲		حالت DIAGNOSTIC_LED_MODE_HEARTBEAT	۲.۵.۱۴

۴۲	 DIAGNOSTIC_LED_MODE_DANCE	حالت	۳.۵.۱۴
۴۲	 DIAGNOSTIC_LED_MODE_ERROR	حالت	۴.۵.۱۴
۴۲		اتصال به چرخه برنامه	۶.۱۴
۴۳		پروگرام با رابط چهار سیمه	۷.۱۴
۴۳		نتیجه پروگرام انجام شده	۸.۱۴
۴۳		روند آزمون و تفسیر نتیجه	۹.۱۴
۴۵	۱۵ راه اندازی ارتباط و اندازه گیری با ADE7880		
۴۵		دامنه این مرحله و نتیجه قابل انتظار	۱.۱۵
۴۵		منبع بازیابی اتصال های قبلی	۲.۱۵
۴۶		کنترل الزامات دیتاشیت	۳.۱۵
۴۷	 STM32CubeMX	تنظیم از صفر در	۴.۱۵
۴۸		معماری کتابخانه قابل استفاده مجدد	۵.۱۵
۴۸		قرارداد کامنت گذاری و روش خواندن کد	۶.۱۵
۴۹		مرجع سریع توابع کتابخانه	۷.۱۵
۵۰		توالی راه اندازی برنامه	۸.۱۵
۵۰		رجیسترها و قالب داده خوانده شده	۹.۱۵
۵۱	 PlatformIO و VS Code	خواندن در	۱۰.۱۵
۵۲	 Keil MDK	ساخت و دیباگ در	۱۱.۱۵
۵۲		کالیبراسیون دو نقطه ای	۱۲.۱۵
۵۳		ضرایب ولتاژ بازیابی شده و خروجی فعلی	۱.۱۲.۱۵
۵۴		مشاهده جریان، توان و ضریب توان در دیباگر	۲.۱۲.۱۵
۵۵	 Watch	کالیبراسیون جریان و توان برای خروجی	۳.۱۲.۱۵
۵۶		آزمون سخت افزاری مرحله به مرحله	۱۳.۱۵
۵۶		آزمون بدون اعمال ولتاژ اندازه گیری	۱.۱۳.۱۵

۵۷	۲.۱۳.۱۵ آزمون با منبع مرجع
۵۷	۱۴.۱۵ راهنمای عیب‌یابی
۵۸	۱۵.۱۵ نتیجه اعتبارسنجی نرم‌افزاری این مرحله
۵۹	۱۶.۱۵ چک‌لیست پذیرش این قابلیت

فصل ۱

دامنه، هدف و نمای کلی سامانه

۱.۱ کاربرد

برد کنترل برای یک اتاق آزمون تجهیزات برودتی ساخته شده است. دو ایستگاه اندازه‌گیری، در مجموع ۳۲ حسگر دمای دیجیتال را می‌خوانند؛ دو حسگر دمای محیط روی دو گذرگاه مستقل نیز مرجع کنترل اتاق‌اند. رطوبت محیط از خروجی آنالوگ حسگر خانواده HIH نمونه‌برداری می‌شود. مدار ADE7880 کمیت‌های الکتریکی دو کانال فعال را اندازه می‌گیرد. رایانه از طریق RS-232 داده‌ها را دریافت و رله‌ها، سرمایش، گرمایش، رطوبت‌ساز و نقطه‌های تنظیم را کنترل می‌کند.

۲.۱ حدود نسخه حاضر

این نسخه قابلیت تازه‌ای به محصول اضافه نمی‌کند. هدف آن حفظ رفتار شناخته‌شده، حذف خطرهای قطعی، جداسازی کد کاربردی از کد مولد، فراهم کردن ساخت با دو زنجیره ابزار و ایجاد مرجع نگهداری است. دو ایستگاه سوم و چهارم در قالب مخابراتی رزرو شده‌اند، ولی سخت‌افزار و منطق فعال ندارند.

۳.۱ نمای جریان داده

ترتیب منطقی اجرا چنین است:

۱. وقفه‌های تایمر تنها موعداً اجرای کار را ثبت می‌کنند.
۲. حلقه اصلی کمیت الکتریکی، رطوبت و دما را در موعداً مربوط می‌خواند.

۳. کنترل خودکار با هیستریزیس و پنجره زمانی هیتر خروجی ها را تنظیم می کند.
۴. یک قاب ثابت ۲۰۹ بایتی ساخته و روی درگاه سریال ارسال می شود.
۵. دریافت سریال به صورت وقفه ای و تک بایتی انجام و فرمان کامل در حلقه اصلی تفسیر می شود.

۴.۱ واحدهای مهم

دمای محیط در داخل برنامه با واحد یکدهم درجه سلسیوس نگهداری می شود. داده ۳۲ حسگر ایستگاه ها برای سازگاری با نرم افزار قبلی به شکل مقدار خام حسگر، یعنی واحد یک شانزدهم درجه، ارسال می شود. رطوبت ارسالی یکدهم درصد و مقادیر چندبایتی همگی کم ارزش ترین بایت در ابتدا هستند.

فصل ۲

سخت افزار و تنظیمات مولد کد

هشدار: جدول کامل پیرامونی‌های این فصل معماری محصول پیش از پاک‌سازی را ثبت می‌کند. در snapshot فعال این مرحله فقط کلاک سامانه، SWD، دو LED و SPI2/ADE7880 دوباره فعال شده‌اند؛ مراحل دقیق بازسازی فعلی در فصل ۱۵ است.

۱.۲ پردازنده و کلاک

قطعه انتخاب‌شده STM32F107VCT6 در بسته LQFP100 است. نوسان‌ساز خارجی ۸ مگاهرتز با ضریب ۹، کلاک سامانه ۷۲ مگاهرتز می‌سازد. کلاک گذرگاه اول ۳۶ و گذرگاه دوم ۷۲ مگاهرتز است. کلاک مبدل آنالوگ ۱۲ مگاهرتز تنظیم شده است.

۲.۲ پیرامونی‌های فعال

پیرامونی	تنظیم و کاربرد
ADC1	کانال‌های ۵ و ۴، اسکن پیوسته، زمان نمونه‌برداری ۵.۲۳۹ سیکل و انتقال چرخشی با کانال اول DMA1؛ کانال نخست منبع فعال رطوبت است.
SPI2	حالت ارباب، هشت‌بیتی، قطبیت بالا، لبه دوم، تقسیم کلاک ۶۴؛ ارتباط با ADE7880 و انتخاب تراشه روی پایه PB12.

تنظیم و کاربرد	پیرامونی
نرخ ۹۶۰۰، هشت بیت، یک توقف، بدون توازن، ارسال و دریافت؛ پایه‌های بازنگاشت شده PD5/PD6.	USART2
نرخ ۱۰۰ کیلوهرتز و بازنگاشت روی PB8/PB9؛ برای کتابخانه حسگر AM2315 آماده است ولی در مسیر فعال استفاده نمی‌شود.	I2C1
دوره یک ثانیه؛ موعد ارسال داده.	TIM1
دوره نیم ثانیه؛ موعد انرژی، رطوبت، شمارنده استارت و پنجره هیتر.	TIM2
دوره یک ثانیه؛ موعد خواندن حسگرهای دما.	TIM3

۳.۲ پایه‌های کاربردی قطعی

پایه	نقش	نکته
PC6	گذرگاه نخست 1-Wire	شانزده حسگر ایستگاه اول و حسگر محیط اول
PC7	گذرگاه دوم 1-Wire	شانزده حسگر ایستگاه دوم و حسگر محیط دوم
PA5	ورودی آنالوگ نخست	ورودی فعال محاسبه رطوبت
PA4	ورودی آنالوگ دوم	نمونه‌برداری می‌شود ولی فعلاً مصرف کاربردی ندارد
PE1	رطوبت‌ساز	فرمان فعال پایین
PE2	رله ایستگاه اول	فرمان دستی فعال پایین
PE4,PE5	دو خروجی سرمایش	هم‌زمان کنترل می‌شوند و فعال پایین‌اند
PB3	هیتر یا SSR	فعال پایین و دارای کنترل پنجره زمانی
PB12	انتخاب تراشه انرژی	در حالت بی‌کار بالا
PA13,PA14	برنامه‌ریزی و اشکال‌زدایی	رابط دو سیمه SWD برای J-Link؛ زمین و تغذیه مرجع نیز لازم است

پایه	نقش	نکته
PC13, PC14	دو چراغ عیب‌یابی	خروجی پوش‌پول کم‌سرعت، سطح اولیه بالا و الگوی غیرمسدودکننده زنده‌بودن برنامه

پایه‌های خروجی دیگر شامل PE3, PE6, PB4, PD0, PD2, PD3, PD8, PD14, PD15, PC8 در فایل مولد تعریف شده‌اند اما نقش دقیقشان از سورس معلوم نیست. دو پایه PE6 و PD3 مطابق رفتار نسخه قدیمی هنگام توقف کنترل در سطح بالا قرار می‌گیرند. پیش از حذف یا تغییر هر کدام باید شماتیک بررسی شود.

۴.۲ ایمنی تولید مجدد

گزینه نگهداشتن کد کاربر فعال، پشتیبان‌گیری نسخه قبلی فعال و حذف پیشین غیرفعال شده است. مقدار دوره تایمر دوم نیز از اختلاف خطرناک ۳۵ به مقدار عملیاتی ۳۵۹۹ همگام شد. پس از هر تولید مجدد باید تفاوت فایل‌ها بازبینی و وجود فراخوانی‌های App_Init و App_Process در بلوک‌های حفاظت‌شده کنترل شود.

فصل ۳

معماری نرم افزار و مالکیت فایل ها

۱.۳ لایه ها

فایل Core/Src/main.c تنها راه اندازی تولید شده و دو فراخوانی کاربردی را نگه می دارد. فایل Core/Src/stm32f1xx_it.c تنها هندلرهای استاندارد HAL را اجرا می کند. تمام سیاست محصول در Core/Src/app.c قرار دارد و ذخیره پایدار در Core/Src/sensor_address_store.c جدا شده است. درایورهای حسگر و انرژی در فایل های هم نام خود باقی مانده اند.

۲.۳ قرارداد با فایل های تولیدی

تنها تغییرات دستی مجاز در فایل مولد باید بین نشانگرهای USER CODE و USER CODE BEGIN باشد. در نسخه حاضر، سرآیند کاربرد در بلوک شامل ها، ساختار دسته های پیرامونی و فراخوانی راه اندازی در بلوک شماره ۲، و فراخوانی پردازش در حلقه حفاظت شده قرار دارند. منطق تجاری نباید دوباره به فایل وقفه منتقل شود.

۳.۳ زمان بندی تعاونی

هر تایمر در تابع بازگشتی HAL فقط یک پرچم volatile را فعال می کند. حلقه اصلی پرچم ها را به ترتیب فرمان دریافتی، انرژی، دما و ارسال مصرف می کند. اگر یک کار طولانی شود، پرچم رویداد بعدی ادغام می شود؛ بنابراین حافظه سرریز نمی کند، اما نرخ دقیق نمونه برداری ممکن است کاهش یابد. این رفتار برای نسخه بدون سیستم عامل آگاهانه است و باید در آزمون بار سنجیده شود.

۴.۳ همروندی

وقفه دریافت تنها بایت را در بافر ۱۲ بایتی می گذارد. تفسیر فرمان، نوشتن Flash و تغییر خروجی ها در متن اصلی انجام می شود. میانگین گیری ADC تنها عملیات جمع و شمارش کوتاه را در callback انجام می دهد. متغیرهای مشترک ISR و حلقه اصلی volatile تعریف شده اند.

۵.۳ حافظه پایدار

آخرین صفحه دو کیلوبایتی Flash از نشانی 0x0803F800 برای جدول ۳۲ شناسه رزرو است و از ناحیه لینک هر دو زنجیره ساخت حذف شده است. رکورد شامل امضا، نسخه، تعداد، ۲۵۶ بایت شناسه و CRC32 است. هر شناسه نیز با کد خانواده ۲۸ و CRC8 Maxim اعتبارسنجی می شود. در صورت نبود رکورد جدید، جدول قدیمی در 0x08009000 خوانده و در صورت اعتبار به قالب جدید مهاجرت می شود؛ در غیر این صورت جدول پیش فرض کد استفاده می گردد.

فصل ۴

دراپورها، ماژول‌ها و وضعیت استفاده

۱.۴ دراپور دمای دیجیتال

فایل‌های DS18B20.c و DS18B20.h دو گذرگاه نرم‌افزاری روی پایه‌های PC6 و PC7 پیاده می‌کنند. فرمان تطبیق‌شناسه، تبدیل دما و خواندن حافظه موقت استفاده می‌شود. تابع راه‌اندازی تازه، خطوط را پیش از نخستین تراکنش به خروجی open-drain و حالت آزاد تبدیل می‌کند.

خروجی تابع دما همچنان مقدار خام یک‌شانزدهم درجه است تا پروتکل رایانه تغییر نکند. دراپور فعلی حضور حسگر، CRC حافظه موقت و پایان واقعی تبدیل را کنترل نمی‌کند و تأخیر میکروثانیه‌ای آن وابسته به بهینه‌ساز و کلاک پردازنده است. عدد خطای ۹۰۹۵ درجه که از خواندن همه بیت‌های یک ناشی می‌شود، در کد قدیمی با مقدار ۴۰۹۵۹ در واحد یک‌دهم تشخیص داده می‌شود. برای نسخه صنعتی بعدی، تایمر سخت‌افزاری، تبدیل هم‌زمان همه حسگرها و بررسی CRC توصیه می‌شود.

۲.۴ اندازه‌گیری انرژی

ماژول قابل‌حمل ADE7880 روی SPI2 کار می‌کند و وابستگی مستقیم به HAL ندارد. برنامه ولتاژ مؤثر، جریان مؤثر، توان اکتیو، توان ظاهری و ضریب توان هر سه فاز و جریان نول را در snapshot سازگار می‌خواند. ضریب توان از رجیسترهای مستقل APF/BPF/CPF گرفته می‌شود. خطا و timeout به فراخواننده بازمی‌گردند و reset حلقه نامحدود ندارد. شرح کامل اتصال، API، کالیبراسیون و آزمون در فصل ۱۵ آمده است.

۳.۴ رطوبت آنالوگ

کانال فعال ADC پس از هر ۲۰۰۰ نمونه میانگین‌گیری می‌شود. تبدیل فعلی چنین است: ولتاژ مرجع تجربی ۳۳۶.۳ ولت، آفست ۸۵۳.۰ ولت، شیب ۳۰ میلی‌ولت بر درصد و دو ضریب اصلاح ۳.۱ و ۰.۴۱. خروجی برای جلوگیری از فرمان نامعتبر به بازه صفر تا صد درصد محدود شده است. این ضرایب وابسته به برد و کالیبراسیون‌اند و باید در پرونده کالیبراسیون محصول ثبت شوند.

۴.۴ حسگر رطوبت دیجیتال

ماژول AM2315 در پروژه Keil کامپایل می‌شود و I2C1 نیز راه‌اندازی شده است، اما فراخوانی آن در مسیر اجرایی وجود ندارد. پیاده‌سازی بازبینی‌شده کدهای بازگشت HAL، ساختار پاسخ، علامت دما و CRC16 Modbus را بررسی می‌کند. این ماژول برای سازگاری نگه داشته شده، ولی پیش از فعال‌سازی باید روی سخت‌افزار تست شود.

۵.۴ فیلتر میانگین متحرک

ماژول MAF پیاده‌سازی پنجره ۴۰ نمونه‌ای دارد، اما نه در پروژه Keil افزوده شده و نه در هیچ سورس فعال فراخوانی می‌شود. محاسبه آن در هر به‌روزرسانی کل پنجره را دوباره جمع می‌کند. برای استفاده آینده بهتر است جمع جاری عضو ساختار باشد تا هزینه ثابت شود. در نسخه فعلی حذف نشده، زیرا یک کتابخانه مستقل و قابل استفاده است؛ وضعیت غیرفعال آن باید حفظ شود.

۶.۴ درایورهای HAL

ماژول‌های ADC، DMA، GPIO، I2C، SPI، تایمر، UART، Flash، RCC، توان، Cortex و EXTI در تنظیمات و پروژه Keil حضور دارند. EXTI کاربرد مستقیم ندارد ولی وابستگی زیرساختی HAL است. فایل‌های LL موجود در پوشه درایورها کامپایل نمی‌شوند. آثار آبجکت‌هایی مانند ADE7753، MCP4725 و نسخه‌های قدیمی DS18B20 در پوشه خروجی Keil سورس متناظر ندارند و بخشی از محصول فعال نیستند.

فصل ۵

رفتار زمان اجرا و کنترل محیط

۱.۵ راهاندازی

پس از راهاندازی کلاک و پیرامونی‌ها، همه محرک‌های فعال پایین خاموش می‌شوند، گذرگاه‌های دما آزاد می‌شوند، جدول شناسه بارگذاری می‌شود و مدار انرژی به حالت SPI قفل، بازنشانی و اجرا می‌رود. سپس DMA، دریافت تک‌بایتی UART و سه تایمر آغاز می‌شوند. تأخیر پنج‌ثانیه‌ای قدیمی و ارسال I2C با اشاره‌گر نامعتبر حذف شده‌اند.

۲.۵ کنترل رطوبت

در حالت خودکار، پایین‌تر از ۹۸ درصد نقطه تنظیم رطوبت‌ساز روشن و از ۱۰۱ درصد آن به بالا خاموش می‌شود. بین دو مرز وضعیت قبلی حفظ می‌شود. در نسخه قدیمی خرابی حسگر دیجیتال مانع کنترل بود، درحالی‌که حسگر فعال آنالوگ است؛ وابستگی نادرست حذف شده است.

۳.۵ کنترل سرمایش

اگر دمای محیط بیش از نیم درجه بالاتر از نقطه تنظیم باشد، دو خروجی سرمایش روشن می‌شوند. اگر دما نیم درجه یا بیشتر پایین‌تر باشد، خاموش می‌شوند. ناحیه یک‌درجه‌ای میانی هیستریزیس از قطع‌ووصل سریع جلوگیری می‌کند. حفاظت حداقل زمان خاموش یا روشن کمپرسور هنوز وجود ندارد و باید پیش از اتصال مستقیم کمپرسور توسط سخت‌افزار یا نرم‌افزار جداگانه تضمین شود.

۴.۵ کنترل گرمایش

زیر ۹۵ درصد نقطه تنظیم، هیتر پیوسته روشن و از ۱۰۱ درصد به بالا خاموش است. در ناحیه میانی، رابطه تجربی کد قدیمی به یک پنجره صدمرحله‌ای محدود شده است. این امکان روشن ماندن سرمایش و جبران با هیتر را حفظ می‌کند. رابطه تجربی در نقطه تنظیم تقریباً ۱۶ درصد زمان روشن می‌دهد؛ در لبه پایین می‌تواند نزدیک صد درصد شود.

۵.۵ حالت‌ها و ایمنی

فرمان AUTOFF کنترل خودکار را غیرفعال و همه خروجی‌های شناخته‌شده را خاموش می‌کند. فرمان AUTONN آن را فعال می‌کند. فرمان نقطه تنظیم نیز کنترل را مجاز می‌سازد. فرمان‌های دستی مستقیماً خروجی را تغییر می‌دهند، ولی اگر حالت خودکار فعال بماند چرخه بعدی می‌تواند آن را بازنویسی کند؛ نرم‌افزار رایانه باید پیش از کنترل دستی، حالت خودکار را خاموش کند.

هشدار: معنای الکتریکی خروجی‌ها از روی کد «فعال‌پایین» استنتاج شده است. پیش از نخستین پروگرام نسخه بازآرایی‌شده، حالت خاموش واقعی هر رله و SSR با مولتی‌متر بررسی شود.

فصل ۶

پروتکل سریال رایانه

۱.۶ لایه انتقال

رابط روی ۹۶۰۰ بیت‌برثانیه، هشت بیت داده، یک بیت توقف و بدون توازن است. هر فرمان ۱۲ بایت دارد. اگر فرمان کوتاه‌تر است، فرستنده باید با صفر آن را تا ۱۲ بایت پر کند. هیچ مجموع مقابله با خطا، شماره توالی یا گریز از قاب در نسخه قدیمی تعریف نشده است؛ برای کابل نویزی افزودن نسخه دوم پروتکل توصیه می‌شود.

۲.۶ فرمان‌ها

پیشوند	اثر
S1FF / S1NN	خاموش یا روشن کردن رله ایستگاه اول روی PE2.
S2FF / S2NN	خاموش یا روشن کردن خروجی PE1؛ با فرمان‌های رطوبت هم‌پوشان است.
COLF / COLN	خاموش یا روشن کردن هر دو خروجی سرمایش.
HUMF / HUMN	خاموش یا روشن کردن رطوبت‌ساز.
HETF / HETN	خاموش یا روشن کردن هیتر.
SETP	بایت‌های ۴ و ۵ دما و بایت‌های ۶ و ۷ رطوبت، بدون علامت، کم‌ارزش ابتدا و با مقیاس یک‌دهم.

پیشوند	اثر
TMP	بایت ۳ شماره یک‌مبنای حسگر و بایت‌های ۴ تا ۱۱ شناسه ROM؛ بازه و CRC بررسی و سپس کل جدول ذخیره می‌شود.
AUTOFF و AUTONN	غیرفعال یا فعال کردن کنترل خودکار.
EXIT	بازتاب ۱۲ بایت، خاموش‌سازی خروجی‌ها و بازنشانی پردازنده.

۳.۶ قالب داده ۲۰۹ بایتی

بازه بایت	قالب	معنا
۰ تا ۳	ثابت	سرآیند AA BB CC DD
۴	هشت‌بیتی	مقدار ۲۰۴، طول داده پس از این بایت
۵ تا ۱۲	چهار عدد ۱۶ بیتی	ولتاژ، جریان، توان و ضریب توان ایستگاه اول
۱۳ تا ۲۰	چهار عدد ۱۶ بیتی	همان کمیت‌ها برای ایستگاه دوم
۲۱ تا ۳۶	صفر رزرو	انرژی ایستگاه‌های سوم و چهارم
۳۷ تا ۶۸	شانزده عدد ۱۶ بیتی	دماهای خام ایستگاه اول
۶۹ تا ۱۰۰	شانزده عدد ۱۶ بیتی	دماهای خام ایستگاه دوم
۱۰۱ تا ۱۶۴	صفر رزرو	دمای ایستگاه‌های سوم و چهارم
۱۶۵ تا ۱۶۶	۱۶ بیتی	دمای محیط با واحد یک‌دهم درجه
۱۶۷ تا ۱۶۸	۱۶ بیتی	رطوبت محیط با واحد یک‌دهم درصد
۱۶۹ تا ۱۷۲	دو عدد ۱۶ بیتی	شمارنده استارت دو کانال، ضرب‌شده در ده برای سازگاری
۱۷۳ تا ۱۷۶	صفر رزرو	شمارنده ایستگاه‌های سوم و چهارم
۱۷۷ تا ۲۰۴	دنباله ثابت	مقادیر ۷ تا ۳۴ ده‌دهی؛ نقش کاربردی ناشناخته
۲۰۵ تا ۲۰۸	ثابت	پایان FC FD FE FF

تمام اعداد ۱۶ بیتی به ترتیب کم ارزش ترین بایت و سپس پر ارزش ترین بایت اند. آستانه اعتبار ولتاژ ۱۰۰ تا کمتر از ۳۰۰۰ است؛ بیرون این بازه سه کمیت اصلی همان ایستگاه صفر می شوند. ضریب توان فعلاً همواره صفر است.

فصل ۷

نتیجه ممیزی و تغییرات انجام شده

۱.۷ نقاط قوت نسخه اولیه

تنظیمات اصلی کلاک و پیرامونی‌ها روشن و خروجی ساخت Keil موجود موفق و بدون هشدار بوده است. پروتکل ثابت، ضرایب کالیبراسیون عملیاتی، دو گذرگاه دما، شناسه‌های یکتا و هیستریزیس کنترل، دانش ارزشمند محصول را ثبت کرده‌اند. استفاده از نشانگرهای حفاظت‌شده CubeMX در بخش‌هایی از فایل اصلی نیز پایه مناسبی بوده است.

۲.۷ اشکال‌های بحرانی اصلاح شده

- عملیات طولانی SPI، 1-Wire و UART از متن وقفه خارج شد.
- خواندن مستقیم ثبات UART پس از HAL و فراخوانی دوباره هندلر حذف و دریافت استاندارد وقفه‌ای جایگزین شد.
- منطق کاربردی از فایل وقفه تولیدشده به ماژول مستقل انتقال یافت.
- اختلاف دوره تایمر دوم میان فایل مولد و سورس برطرف شد.
- خواندن از 0x08009000 و نوشتن در 0x08008000 با یک صفحه رزرو شده، نسخه‌دار و دارای CRC جایگزین شد.
- حلقه‌ای که ۶۴ مقدار را از آرایه ۳۲ عضوی می‌خواند حذف شد.
- خطر هم‌پوشانی داده تنظیمات با کد برنامه با کاهش ناحیه لینک برطرف شد.

- بلوک‌های بزرگ کد کامنت شده و متغیرهای بایت به بایت تکراری حذف و ساخت قاب حلقه‌ای شد.
- تقسیم بر صفر در محاسبه پنجره هیترو و خروجی خارج از بازه رطوبت مهار شد.
- حالت خاموش خودکار اکنون واقعاً پایدار و ایمن است.

۳.۷ موارد حفظ شده

قالب ۲۰۹ بایتی، نرخ سریال، نگاشت دو ایستگاه، ترتیب بایت، آدرس‌های پیش فرض، ضرایب کالیبراسیون، منطق فعال پایین، هیستریزیس و رابطه پنجره هیترو حفظ شده‌اند. جابه‌جایی ظاهری شمارنده‌های استارت نیز برای سازگاری قاب قدیمی حفظ و در کد روشن ثبت شده است.

۴.۷ ریسک‌های باز

۱. درایور 1-Wire هنوز زمان تبدیل کافی مطابق بدترین حالت دیتاشیت و CRC داده را تضمین نمی‌کند.
۲. تبدیل‌های طولانی ۳۴ حسگر ترتیبی‌اند و می‌توانند نرخ حلقه را کاهش دهند.
۳. پروتکل ورودی همگام‌سازی، CRC، پاسخ موفقیت و پاسخ خطا ندارد.
۴. حفاظت زمانی کمپرسور، تشخیص اتصال کوتاه حسگر و واچ‌داگ فعال نیست.
۵. نقش چند خروجی GPIO، کانال دوم ADC و دنباله بایت‌های ۱۷۷ تا ۲۰۴ مستند سخت‌افزاری ندارد.
۶. ضرایب انرژی و رطوبت تنها از کد استخراج شده‌اند و شناسنامه کالیبراسیون همراه پروژه نیست.

۵.۷ چرا بعضی فایل‌ها حذف نشدند

کتابخانه‌های AM2315 و MAF سورس مستقل و بالقوه مفیدند و حذفشان برای پروژه‌های بعدی برگشت‌پذیری را کم می‌کند؛ بنابراین با برچسب غیرفعال نگه داشته شدند. در مقابل، خروجی‌های ساخت قدیمی در این مرحله فیزیکی حذف نشدند، چون مسیر فعلی مخزن Git نیست و ممکن است برای مقایسه باینری لازم باشند؛ فایل ignore مانع ورود دوباره آن‌ها به کنترل نسخه خواهد شد.

فصل ۸

ساخت، پروگرام و تولید مجدد

۱.۸ ساخت با Keil

پروژه MDK-ARM/STM32F107.uvprojx را باز کنید، بسته خانواده STM32F1 را در Keil نصب کنید و هدف را بسازید. دو فایل کاربرد و ذخیره‌سازی به گروه کاربر افزوده شده‌اند. اندازه IROM1 برابر 0x3F800 است و نباید به ۲۵۶ کیلوبایت کامل بازگردد. پروگرامر J-Link از SWDIO, SWCLK, GND و ولتاژ مرجع برد استفاده می‌کند.

۲.۸ ساخت با PlatformIO

در ریشه پروژه اجرا کنید:

```
pio run
pio run --target upload
pio device monitor
```

محیط stm32f107vc_jlink از شناسه رسمی برد eval_f107vc، چارچوب STM32Cube و J-Link استفاده می‌کند. فایل لینک اختصاصی آخرین صفحه Flash را رزرو می‌کند. فایل سامانه محلی از ساخت PlatformIO کنار گذاشته می‌شود تا نسخه چارچوب و فایل پروژه هم‌زمان نماد تکراری نسازند.

۳.۸ تولید مجدد با CubeMX

۱. پیش از تولید، از پوشه پروژه نسخه پشتیبان یا یک commit بسازید.
۲. فایل STM32F107.ioc را باز و گزینه حفظ کد کاربر را کنترل کنید.
۳. دوره تایمر دوم باید ۳۵۹۹ و کلاک تایمر ۷۲ مگاهرتز باشد.
۴. کد Keil را تولید کنید و هیچ اخطار حذف فایل را بدون بازبینی نپذیرید.
۵. بررسی کنید فایل‌های app.c و sensor_address_store.c هنوز عضو پروژه Keil باشند.
۶. اندازه Flash لینک Keil را دوباره روی 0x3F800 قرار دهید؛ مولد ممکن است آن را بازنویسی کند.
۷. تفاوت‌ها را بررسی، سپس هر دو ساخت را اجرا کنید.

هشدار: هیچ فایل HEX قدیمی را پس از تغییر سورس به عنوان نتیجه معتبر تلقی نکنید. زمان و خروجی ساخت باید با سورس همان نسخه ثبت شود.

۴.۸ ساخت گزارش

از پوشه docs دستور زیر را دوبار اجرا کنید تا فهرست کامل شود:

```
xelatex -interaction=nonstopmode relay_on_duration_report.tex
```

```
xelatex -interaction=nonstopmode relay_on_duration_report.tex
```

اگر فونت بی‌نازنین نصب باشد خودکار استفاده می‌شود و در غیر این صورت تاهوما جایگزین است. رنگ‌ها، حاشیه‌ها و فونت‌ها تنها در فایل تنظیمات متمرکز قرار دارند.

فصل ۹

تأیید، تحویل و نگهداری آینده

۱.۹ آزمون رومیزی بدون بار قدرت

ابتدا رله‌ها و SSRها را از بار اصلی جدا کنید. سطح خاموش همه خروجی‌ها، کلاک ۷۲ مگاهرتز، نرخ UART و انتخاب تراشه SPI را با ابزار اندازه‌گیری کنترل کنید. سپس یک حسگر روی هر گذرگاه وصل و شناسه، دمای خام و دمای محیط را با دماسنج مرجع مقایسه کنید.

۲.۹ آزمون پروتکل

برای هر فرمان یک قاب دقیق ۱۲ بیتی بفرستید و اثر خروجی را ثبت کنید. قاب ۲۰۹ بیتی را از نظر سرآیند، انتها، طول، ترتیب کم‌ارزش‌ترین بایت و جایگاه هر حسگر بررسی کنید. سناریوهای قطع کابل، بایت گمشده، فرمان نامعتبر و توقف نرم‌افزار رایانه نیز باید آزموده شوند.

۳.۹ آزمون کنترل

با بار شبیه‌ساز، نقاط پایین، داخل و بالای هیستریزیس دما و رطوبت را اعمال کنید. گذار خودکار به دستی و برعکس، بازنشانی، قطع حسگر و بازیابی برق را ثبت کنید. شمارنده استارت باید تنها روی لبه خاموش به روشن افزایش یابد. سپس حداقل زمان مجاز کمپرسور طبق سازنده به آزمون اضافه شود.

۴.۹ معیار پذیرش نسخه

- ساخت Keil و PlatformIO بدون خطا انجام شود.
- اندازه کد از مرز 0x0803F800 عبور نکند.
- تولید مجدد CubeMX هیچ کد کاربردی را حذف نکند.
- خاموشی ایمن هنگام راه اندازی، خطا و AUTOFF تأیید شود.
- ۳۲ شناسه پس از قطع برق سالم بازیابی و CRC آن ها تأیید شود.
- نرم افزار رایانه قاب جدید را بدون تغییر تفسیر کند.

۵.۹ اولویت های توسعه بعدی

اولویت نخست بازنویسی غیرمسدودکننده 1-Wire با تبدیل همزمان و CRC است. سپس پروتکل نسخه دار با CRC و پاسخ، ماشین حالت حفاظت کمپرسور، واچ داگ، گزارش خطای حسگر و تست واحد میزبان پیشنهاد می شود. پس از دریافت شماتیک می توان نام و وضعیت پایه های ناشناخته را قطعی و مازول های بلااستفاده را حذف یا فعال کرد.

۶.۹ قاعده نگهداری سند

هر تغییر رفتاری باید همزمان در فصل مربوط، فایل وضعیت ریشه، شماره نسخه پروتکل و فهرست آزمون ثبت شود. نتیجه آزمون سخت افزاری نباید از روی موفقیت کامپایل استنتاج شود. فایل وضعیت پروژه نقطه شروع هر عامل انسانی یا هوش مصنوعی برای ادامه کار است.

فصل ۱۰

مرجع کامل توابع لایه کاربرد

این فصل فایل `Core/Src/app.c` را از دید توسعه‌دهنده شرح می‌دهد. توابع ایستا فقط در همان فایل قابل استفاده‌اند؛ توابع عمومی از طریق `Core/Inc/app.h` در اختیار فایل تولیدشده قرار می‌گیرند.

۱.۱۰ ساختار ورودی راه‌اندازی

ساختار `App_Peripherals` مالک سخت‌افزار نیست و فقط اشاره‌گر دسته‌های ساخته‌شده در فایل اصلی را نگه می‌دارد. اعضای آن عبارت‌اند از دسته مبدل آنالوگ، رابط رزرو شده رطوبت دیجیتال، رابط انرژی، سه تایمر زمان‌بندی و رابط سریال. طول عمر این دسته‌ها سراسری است؛ بنابراین اشاره‌گرها پس از پایان تابع راه‌اندازی معتبر می‌مانند.

۲.۱۰ توابع عمومی چرخه اجرا

۱.۲.۱۰ تابع `App_Init`

ورودی تابع، اشاره‌گر ثابت به ساختار دسته‌های پیرامونی است و خروجی صریح ندارد. تابع ابتدا ساختار را کپی می‌کند، خروجی‌های فعال‌پایین را خاموش می‌سازد، دو خط دما را آزاد می‌کند و جدول شناسه حسگرها را از حافظه می‌خواند. اگر رکورد معتبر نباشد، جدول پیش‌فرض استفاده می‌شود. سپس مدار انرژی به رابط متصل، بازنشانی و اجرا می‌شود و در پایان انتقال چرخشی مبدل آنالوگ، دریافت تک‌بایتی سریال و سه تایمر شروع می‌شوند.

اثرهای جانبی تابع شامل تغییر پایه‌ها، تراکنش روی `SPI`، خواندن احتمالی `Flash` و فعال‌سازی وقفه‌هاست.

این تابع باید دقیقاً یک بار و پس از تمام توابع راهاندازی مولد فراخوانی شود. ورودی تهی یا دسته نامعتبر در نسخه فعلی کنترل دفاعی نشده، زیرا قرارداد آن با فایل اصلی ثابت است.

۲.۲.۱۰ تابع `App_Process`

این تابع ورودی و خروجی ندارد و باید بدون توقف در حلقه اصلی اجرا شود. ترتیب رسیدگی آن فرمان سریال، نمونه الکتریکی، نمونه دما و ارسال قاب است. هر کار فقط زمانی اجرا می‌شود که `callback` مربوط پرچمش را فعال کرده باشد. مزیت این طراحی آن است که هیچ تراکنش طولانی داخل وقفه رخ نمی‌دهد. محدودیت آن ادغام چند رویداد هم‌نوع است؛ اگر یک کار بیش از دوره تایمر طول بکشد، شمار دقیق رویدادهای عقب‌افتاده حفظ نمی‌شود.

۳.۱۰ توابع داخلی تبدیل و ایمنی

۱.۳.۱۰ تابع `outputs_safe`

ورودی و خروجی ندارد. رطوبت‌ساز، رله ایستگاه اول، دو خروجی سرمایه‌ش و هیتر را با نوشتن سطح بالا خاموش می‌کند. همچنین پایه‌های بدون نام قطعی `PE6` و `PD3` را مطابق رفتار نسخه قدیمی بالا می‌برد. این تابع هنگام راهاندازی و فرمان توقف خودکار استفاده می‌شود.

۲.۳.۱۰ تابع `clamp_u16_from_float`

یک عدد اعشاری می‌گیرد و مقدار قابل ارسال ۱۶ بیتی برمی‌گرداند. مقادیر منفی صفر، مقادیر بیش از ۶۵۵۳۵ برابر بیشینه و مقادیر میانی با حذف بخش اعشاری تبدیل می‌شوند. هدف جلوگیری از سرریز در دما، رطوبت و ضرایب اندازه‌گیری است.

۳.۳.۱۰ تابع `put_u16_le`

ورودی‌ها اشاره‌گر مقصد دو بیتی و مقدار ۱۶ بیتی‌اند. بایت کم‌ارزش در خانه صفر و بایت پرارزش در خانه یک نوشته می‌شود. خروجی صریح ندارد و قرارداد قدیمی رایانه را یکجا اعمال می‌کند.

۴.۱۰ اندازه‌گیری و محاسبه

۱.۴.۱۰ تابع `sample_electrical`

ورودی و خروجی مستقیم ندارد. ابتدا رجیسترهای ولتاژ مؤثر، جریان مؤثر و توان مدار انرژی را می‌خواند. فازهای نخست و دوم به ایستگاه‌های یک و دو نگاشت می‌شوند. ضرایب کالیبراسیون روی داده خام اعمال و نتیجه به ۱۶ بیت محدود می‌شود. اگر ولتاژ خارج از بازه ۱۰۰ تا ۲۹۹۹ باشد، همه کمیت‌های ارسالی آن ایستگاه صفر می‌شوند.

این تابع عبور جریان از آستانه چهار واحد صحیح را نیز برای تشخیص لبه استارت بررسی می‌کند، موقعیت پنجره صدمرحله‌ای هیتر را یک واحد جلو می‌برد و میانگین ADC را با رابطه کالیبراسیون به رطوبت صفر تا صد درصد تبدیل می‌کند. تابع در متن اصلی و تقریباً هر نیم‌ثانیه اجرا می‌شود.

۲.۴.۱۰ تابع `read_temperature_deci_c`

ورودی نخست شماره گذرگاه است؛ صفر گذرگاه PC6 و هر مقدار دیگر گذرگاه PC7 را انتخاب می‌کند. ورودی دوم شناسه هشت‌بایتی حسگر است. تابع مقدار خام یک‌شانزدهم درجه را می‌خواند، بر ۶.۱ تقسیم می‌کند و دما را با واحد یک‌دهم درجه و نوع بدون علامت برمی‌گرداند. در نتیجه نمایش دمای منفی محیط در معماری فعلی کامل نیست و باید در توسعه بعدی اصلاح شود.

۳.۴.۱۰ تابع `sample_temperatures`

دو حسگر محیط روی دو گذرگاه را می‌خواند. مقدار ۴۰۹۵۹ در واحد یک‌دهم به‌عنوان پاسخ قطع‌شده شناخته می‌شود؛ اگر هر دو معتبر باشند میانگین، و اگر یکی معتبر باشد همان مقدار انتخاب می‌شود. سپس حسگرهای صفر تا پانزده از گذرگاه اول و شانزده تا سی‌ویک از گذرگاه دوم خوانده و مقدار خامشان ذخیره می‌شود.

در پایان، نسبت دمای محیط به نقطه تنظیم در رابطه تجربی هیتر قرار می‌گیرد. خروجی محاسبه به بازه صفر تا صد محدود و در `heater_on_count` ذخیره می‌شود. نقطه تنظیم صفر، درخواست گرمایش را صفر می‌کند و از تقسیم بر صفر جلوگیری می‌شود.

۵.۱۰ کنترل خروجی‌ها

۱.۵.۱۰ تابع `apply_automatic_control`

اگر کنترل یا حالت خودکار غیرفعال باشد بدون تغییر باز می‌گردد. رطوبت‌ساز زیر ۹۸ درصد نقطه تنظیم روشن و از ۱۰۱ درصد به بالا خاموش می‌شود. سرمایش بالاتر از نیم درجه نسبت به نقطه تنظیم روشن و پایین‌تر از منفی نیم درجه خاموش می‌شود. هیتر زیر ۹۵ درصد نقطه تنظیم پیوسته روشن، از ۱۰۱ درصد به بالا خاموش و در ناحیه میانی با پنجره زمانی کنترل می‌شود. داخل ناحیه‌های هیستریزیس حالت قبلی حفظ می‌شود.

۶.۱۰ ساخت و ارسال داده

۱.۶.۱۰ تابع `transmit_telemetry`

یک آرایه محلی ۲۰۹ بیتی صفرشده می‌سازد. سرآیند، کمیت‌های دو ایستگاه، ۳۲ دما، دما و رطوبت محیط، شمارنده‌های استارت، دنباله رزرو و انتهای قاب را در جایگاه ثابت می‌نویسد. فضاهای ایستگاه سوم و چهارم صفر می‌مانند. در پایان، کل قاب با مهلت ۲۵۰ میلی‌ثانیه به صورت مسدودکننده ارسال می‌شود. مقدار بازگشت HAL فعلاً فقط نادیده گرفته می‌شود و شمارنده خطای ارسال وجود ندارد.

۷.۱۰ دریافت و تفسیر فرمان

۱.۷.۱۰ تابع `command_is`

اشاره‌گر یک رشته ثابت و طول مقایسه را می‌گیرد و نتیجه منطقی تطبیق پیشوند با بافر فرمان را برمی‌گرداند. این تابع پایان رشته را بررسی نمی‌کند و عمداً برای فرمان‌های دودویی با طول معلوم از مقایسه حافظه استفاده می‌کند.

۲.۷.۱۰ تابع `clear_command`

برای جلوگیری از رقابت با وقفه، وقفه‌ها را کوتاه‌مدت می‌بندد، طول و پرچم آمادگی را صفر می‌کند، وقفه‌ها را باز می‌گرداند و سپس محتوای بافر را پاک می‌کند. داده‌ای که هنگام آماده‌بودن فرمان وارد شود عمداً پذیرفته نمی‌شود؛ نرم‌افزار رایانه باید میان فرمان‌ها فاصله مناسب بگذارد.

۳.۷.۱۰ تابع `process_command`

یک قاب کامل دوازده‌بایتی را با زنجیره فرمان‌های شناخته‌شده مقایسه می‌کند. فرمان‌های دستی پایه‌ها را تغییر می‌دهند؛ فرمان نقطه تنظیم دو عدد کوچک‌مبنا را با ضریب یک‌دهم تبدیل می‌کند؛ فرمان شناسه شماره یک‌مبنا و CRC حسگر را کنترل و سپس کل جدول را ذخیره می‌کند؛ فرمان‌های خودکار حالت کنترل را عوض می‌کنند؛ و فرمان خروج، خروجی‌ها را امن و پردازنده را بازنشانی می‌کند. فرمان ناشناخته بدون اثر حذف می‌شود. تابع پاسخ موفقیت یا خطای رسمی ندارد.

۸.۱۰ توابع ورودی از وقفه

۱.۸.۱۰ تابع `App_OnAdcConversionComplete`

دسته ADC تکمیل‌شده را می‌گیرد. اگر نمونه متعلق به ADC برنامه باشد، نمونه نخست DMA به جمع افزوده می‌شود. پس از ۲۰۰۰ اسکن، میانگین صحیح محاسبه و جمع و شمارنده صفر می‌شوند. تابع در زمینه وقفه اجرا می‌شود و عمداً هیچ محاسبه اعشاری ندارد.

۲.۸.۱۰ تابع `App_OnTimerElapsed`

دسته تایمر را می‌گیرد و با مقایسه نمونه سخت‌افزار، یکی از سه پرچم موعد ارسال، انرژی یا دما را فعال می‌کند. عملیات اصلی را اجرا نمی‌کند و زمان اجرای آن ثابت و کوتاه است.

۳.۸.۱۰ تابع `App_OnUartRxComplete`

پس از دریافت هر بایت فراخوانی می‌شود. تا زمانی که فرمان قبلی آماده نشده، بایت را در خانه بعدی قرار می‌دهد. با رسیدن طول به دوازده، پرچم فرمان فعال می‌شود. در هر حالت دریافت تک‌بایتی بعدی دوباره مسلح می‌شود.

۴.۸.۱۰ تابع `App_OnUartError`

فقط خطای رابط مربوط به برنامه را رسیدگی می‌کند. پرچم سرریز دریافت را پاک و دریافت بعدی را دوباره فعال می‌کند. نوع و تعداد خطا در نسخه حاضر ثبت نمی‌شود.

۵.۸.۱۰ توابع HAL بازنویسی شده

چهار تابع بازگشتی HAL_ADC_ConvCpltCallback ، HAL_TIM_PeriodElapsedCallback ، HAL_UART_RxCpltCallback و HAL_UART_ErrorCallback نقطه اتصال کتابخانه HAL به لایه کاربردند. هرکدام فقط آرگومان را به تابع متناظر App_On... می سپارند. تعریف آن‌ها در فایل مستقل باعث می شود تولید مجدد فایل وقفه، منطق کاربردی را حذف نکند.

۹.۱۰ داده‌های سراسری مهم

جدول حسگرها، آخرین ۳۲ دمای خام، دو ساختار الکتریکی، میانگین و جمع ADC، نقاط تنظیم، حالت کنترل، پنجره هیتر، شمارنده استارت، بافر فرمان و پرچم‌های زمان‌بندی همگی ایستای فایل‌اند. این محدودیت دامنه مانع وابستگی مستقیم سایر ماژول‌ها به حالت داخلی می شود. متغیرهایی که میان ISR و حلقه اصلی مشترک‌اند با volatile علامت‌گذاری شده‌اند؛ سایر داده‌ها فقط در متن اصلی تغییر می کنند.

فصل ۱۱

مرجع کامل توابع درایورها

۱.۱۱ ذخیره شناسه حسگر

۱.۱.۱۱ تابع `crc8_maxim`

ورودی آن اشاره گر داده و طول است و CRC هشت بیتی بازتاب یافته با چند جمله ای 0x8C را برمی گرداند. تابع فقط داخلی است و برای هفت بایت نخست شناسه یک سیمه استفاده می شود.

۲.۱.۱۱ تابع `crc32_record`

روی تمام بایت های رکورد به جز فیلد CRC نهایی اجرا می شود. مقدار آغازین همه یک، چند جمله ای بازتاب یافته 0xEDB88320 و خروجی معکوس شده است. هدف تشخیص رکورد ناقص، فرسوده یا متعلق به نسخه دیگر است.

۳.۱.۱۱ تابع `SensorAddressStore_IsValidRom`

یک شناسه هشت بیتی می گیرد. اشاره گر تهی، کد خانواده غیر از ۲۸ یا CRC نامنطبق نتیجه نادرست می دهد. این تابع هم برای فرمان جدید رایانه و هم اعتبارسنجی کل جدول استفاده می شود.

۴.۱.۱۱ تابع `table_is_valid`

تمام ۳۲ ردیف جدول را به تابع اعتبارسنجی شناسه می‌دهد و با نخستین خطا بازمی‌گردد. سیاست فعلی جدول ناقص را نمی‌پذیرد؛ بنابراین تعویض یک حسگر تنها وقتی ذخیره می‌شود که همه ردیف‌های دیگر نیز معتبر باشند.

۵.۱.۱۱ تابع `SensorAddressStore_Load`

رکورد صفحه پایانی Flash را بدون کپی اولیه بررسی می‌کند. امضا، نسخه، تعداد، CRC رکورد و CRC تک‌تک شناسه‌ها باید درست باشند. در موفقیت، جدول به خروجی کپی می‌شود. در شکست، جدول خام قدیمی بررسی می‌شود و در صورت اعتبار، علاوه بر کپی خروجی به قالب جدید ذخیره می‌شود. اگر هر دو نامعتبر باشند نتیجه نادرست است و لایه کاربرد از پیش‌فرض‌ها استفاده می‌کند.

۶.۱.۱۱ تابع `SensorAddressStore_Save`

ابتدا کل جدول ورودی را اعتبارسنجی و یک رکورد محلی می‌سازد. سپس Flash را باز، دقیقاً یک صفحه را پاک و رکورد را کلمه‌به‌کلمه می‌نویسد و در پایان Flash را قفل می‌کند. هر خطای HAL نتیجه نادرست می‌دهد. چون پاک کردن صفحه برگشت‌ناپذیر و نوشتن مسدودکننده است، تابع فقط در حلقه اصلی فراخوانی می‌شود.

۲.۱۱ درایور ADE7880

هشدار: این بخش رفتار نسخه قدیمی درایور را برای سابقه بازآرایی ثبت می‌کند. مرجع API فعال و اصلاح‌شده، فصل ۱۵ و فایل `Libraries/ADE7880/Inc/ade7880.h` است.

۱.۲.۱۱ تابع `ADE7880_ReadRegister`

ورودی‌ها نمونه درایور، نشانی ۱۶ بیتی رجیستر، تعداد بایت و مهلت HAL هستند. بافر چهاربایتی داخلی صفر، فرمان خواندن و نشانی ارسال، سپس یک تا چهار بایت دریافت می‌شود. انتخاب تراشه پیرامون کل تراکنش پایین و بالا می‌رود. داده در TEMP نمونه درایور قرار می‌گیرد و وضعیت خطای HAL هنوز به فراخواننده بازگردانده نمی‌شود.

۲.۲.۱۱ تابع ADE7880_WriteRegister

داده ۳۲بیتی را متناسب با طول یک، دو یا چهار بایت با ترتیب پرارزش‌ترین بایت آماده می‌کند، فرمان نوشتن و نشانی را پیشوند می‌دهد و تراکنش SPI را زیر انتخاب تراشه انجام می‌دهد. طول‌های دیگر نباید ارسال شوند و در نسخه آینده بهتر است نوع نتیجه برای خطا افزوده شود.

۳.۲.۱۱ تابع ADE7880_ConfigSPI

دسته SPI و پایه انتخاب تراشه را در نمونه درایور ذخیره می‌کند. سه بار انتخاب تراشه را تغییر می‌دهد تا رابط سریال مدار روی SPI قرار گیرد، سپس رجیستر پیکربندی دوم را می‌نویسد تا انتخاب قفل شود. پس از بازنشانی سخت‌افزاری باید دوباره اجرا شود.

۴.۲.۱۱ تابع ADE7880_Run

رجیستر اجرای پردازشگر داخلی مدار را با مقدار یک می‌نویسد. ورودی فقط نمونه درایور است و پس از ارسال یک میلی‌ثانیه صبر می‌کند.

۵.۲.۱۱ تابع ADE7880_ReadVRMS

سه رجیستر ولتاژ مؤثر فازهای A تا C را به ترتیب می‌خواند. هر مقدار ۲۴بیتی از سه بایت پایانی بافر ساخته و در ساختار فاز ذخیره می‌شود. تبدیل به ولت در این تابع انجام نمی‌شود.

۶.۲.۱۱ تابع ADE7880_ReadIRMS

مانند تابع ولتاژ است، با این تفاوت که سه جریان فاز و جریان نول را تازه می‌کند. خروجی‌ها هنوز شمار خام رجیستر هستند.

۷.۲.۱۱ تابع ADE7880_ReadPowerz

توان حقیقی و توان ظاهری هر سه فاز را از شش رجیستر ۳۲بیتی می‌خواند. داده در اعضای WATT و VA قرار می‌گیرد. علامت توان حقیقی باید مطابق قالب دیتاشیت تفسیر شود؛ ساختار از نوع علامت‌دار استفاده می‌کند.

۸.۲.۱۱ تابع ADE7880_ReadFPower

چهار رجیستر توان حقیقی، راکتیو، ظاهری و ضریب توان بنیادی را می‌خواند و براساس ورودی انتخاب فاز در ساختار A، B یا C می‌گذارد. این رجیسترها اشتراکی‌اند؛ بنابراین فراخواننده باید پیش‌تر بیت انتخاب فاز رجیستر پیکربندی را تنظیم کرده باشد. برنامه فعلی این تابع را استفاده نمی‌کند.

۹.۲.۱۱ تابع ADE7880_Reset

بیت بازنشانی نرم‌افزاری را در رجیستر پیکربندی می‌نویسد و همان رجیستر را تا صفرشدن بیت بررسی می‌کند. حلقه مهلت کلی ندارد؛ قطع ارتباط SPI می‌تواند راه‌اندازی را متوقف کند. افزودن شمارنده تلاش یکی از اصلاحات ضروری آینده است.

۳.۱۱ درایور دوگانه DS18B20

۱.۳.۱۱ تابع DS18B20_Init

هر دو پایه را خروجی تخلیه‌باز با حالت آزاد بالا می‌کند. این تابع باید پیش از نخستین پالس بازنشانی اجرا شود تا پایه پوش‌پول خط یک‌سیمه را در سطح بالا درایو نکند.

۲.۳.۱۱ توابع زمان و شمار

تابع delayus با یک حلقه وابسته به کلاک تقریباً به‌اندازه ورودی میکروثانیه مکث می‌کند. توابع LH_signal و LH_signal_1 بخش پایین و آزاد هر شمار را روی گذرگاه متناظر تولید می‌کنند. این روش در برابر تغییر سطح بهینه‌سازی یا کلاک حساس است.

۳.۳.۱۱ توابع نوشتن

توابع write_bit و write_bit_1 براساس صفر یا یک، طول بخش پایین شمار را انتخاب می‌کنند. توابع write_byte و write_byte_1 هشت بیت را از کم‌ارزش‌ترین بیت می‌فرستند. ورودی بایت پس از ارسال تغییر نمی‌کند.

۴.۳.۱۱ توابع تغییر جهت پایه

توابع داخلی B9_as_INPUT و C7_as_INPUT خط متناظر را برای نمونه‌برداری ورودی می‌کنند. توابع B9_as_OUTPUT و C7_as_OUTPUT آن را به خروجی تخلیه‌باز بازمی‌گردانند. نام نخست تاریخی است و با پایه واقعی PC6 هم‌خوان نیست؛ تغییر نام باید در یک اصلاح مستقل انجام شود.

۵.۳.۱۱ توابع خواندن

توابع read_bit و read_bit_1 شیار خواندن ایجاد، پایه را ورودی، سطح را نمونه و سپس خروجی می‌کنند. توابع read_byte و read_byte_1 هشت نمونه را از کم‌ارزش‌ترین بیت در یک بایت جمع می‌کنند.

۶.۳.۱۱ توابع انتظار و بازنشانی

توابع B9_wait_for_1 و C7_wait_for_1 گذرگاه را برای مدت ورودی آزاد می‌کنند. توابع reset و reset_1 پالس ۵۰۰ میکروثانیه‌ای پایین و ۵۰۰ میکروثانیه آزاد می‌سازند، اما حضور حسگر را نمونه‌برداری یا گزارش نمی‌کنند.

۷.۳.۱۱ توابع انتخاب شناسه

تابع get_ID فرمان خواندن ROM را روی گذرگاه اول اجرا و هشت بایت را در بافر داخلی قرار می‌دهد؛ فقط زمانی معتبر است که دقیقاً یک حسگر روی گذرگاه باشد. توابع send_ID و send_ID_1 پس از بازنشانی فرمان تطبیق ROM و هشت بایت نشانی ورودی را می‌فرستند.

۸.۳.۱۱ توابع خواندن دما

توابع get_temperature_with_ID و get_temperature_with_ID_1 حسگر را انتخاب، فرمان تبدیل را ارسال، مدت کوتاهی خط را آزاد، دوباره حسگر را انتخاب و نه بایت حافظه موقت را می‌خوانند. دو بایت نخست به عدد خام علامت‌دار تبدیل می‌شوند. CRC حافظه موقت، بیت پایان تبدیل و حضور حسگر هنوز بررسی نمی‌شوند؛ بنابراین خروجی تابع تضمین صحت ندارد.

۴.۱۱ درایور AM2315

۱.۴.۱۱ تابع am2315_crc16

تابع داخلی، CRC شانزدهبیتی بازتابیافته مودباس را با مقدار آغازین 0xFFFF و چندجمله‌ای 0xA001 محاسبه می‌کند. ورودی داده و طول و خروجی CRC است.

۲.۴.۱۱ تابع TEMP_HUM

چهار ورودی آن دسته I2C، بافر فرمان سه‌بایتی، بافر پاسخ هشت‌بایتی و آرایه دو عدد اعشاری خروجی است. ابتدا حسگر را با مرحله نشانی بیدار، سپس فرمان خواندن چهار رجیستر را ارسال و پاسخ را دریافت می‌کند. کد تابع، طول، CRC، علامت دما و وضعیت HAL بررسی می‌شوند. در موفقیت رطوبت و دما نوشته و رطوبت بازگردانده می‌شود؛ در شکست هر دو خروجی صفر و پرچم خطا یک می‌شود. این ماژول در مسیر فعال فعلی فراخوانی نمی‌شود.

۵.۱۱ فیلتر MAF

۱.۵.۱۱ تابع MAF_Init

اشاره‌گر یک نمونه فیلتر را می‌گیرد، معکوس طول پنجره را محاسبه، میانگین و همه ۴۰ خانه را صفر و اشاره‌گر نوشتن را روی خانه نخست قرار می‌دهد. باید پیش از به‌روزرسانی فراخوانی شود.

۲.۵.۱۱ تابع MAF_Update

نمونه جدید را در خانه جاری می‌نویسد و اشاره‌گر را حلقوی جلو می‌برد. سپس همه ۴۰ خانه را جمع و میانگین را ذخیره و برمی‌گرداند. ورودی دوم داده جدید است. هزینه هر فراخوانی خطی است و نمونه‌های صفر اولیه در شروع روی نتیجه اثر دارند.

۳.۵.۱۱ تابع Average

اشاره‌گر آرایه و تعداد اعضا را می‌گیرد و میانگین حسابی برمی‌گرداند. اندازه صفر کنترل نشده و باعث تقسیم بر صفر می‌شود؛ فراخواننده موظف است اندازه مثبت بدهد. این تابع و کل ماژول در برنامه فعلی استفاده نمی‌شوند.

فصل ۱۲

شرح جزئی بازآرایی و مرز تغییرات

۱.۱۲ وضعیت پیش از بازآرایی

فایل وقفه علاوه بر هندلرهای مولد، مالک تقریباً همه حالت‌های برنامه، ۲۰۹ متغیر پراکنده قاب، خواندن ۳۴ حسگر، تراکنش‌های انرژی، محاسبات اعشاری، ارسال مسدودکننده و تفسیر فرمان بود. فایل اصلی نیز آدرس‌ها، کنترل خروجی و callback مبدل آنالوگ را نگه می‌داشت. این پراکندگی وابستگی دوطرفه با تعداد زیادی extern ساخته و تولید مجدد CubeMX را پرخطر کرده بود.

۲.۱۲ انتقال مالکیت

تمام داده‌های محصول به متغیرهای ایستای `app.c` منتقل شدند. فایل اصلی اکنون فقط دسته‌های HAL را در یک ساختار می‌گذارد. فایل وقفه فقط هندلر تولیدشده را صدا می‌زند و callback‌های HAL در فایل کاربرد قرار دارند. در نتیجه جریان وابستگی یک‌طرفه است: فایل مولد، رابط عمومی کاربرد را می‌شناسد؛ کاربرد از دسته‌های تزریق‌شده استفاده می‌کند؛ درایورها از سیاست محصول آگاه نیستند.

۳.۱۲ تبدیل وقفه به کار معوق

پیش‌تر تایمر اول داخل وقفه ۲۰۹ بایت ارسال می‌کرد؛ تایمر دوم همه تراکنش‌های SPI و محاسبه رطوبت را انجام می‌داد؛ تایمر سوم ۳۴ تراکنش دما داشت. اکنون callback تایمر فقط یک پرچم منطقی می‌نویسد و عملیات در حلقه اصلی اجرا می‌شود. این تغییر زمان انسداد وقفه را به چند دستور کاهش می‌دهد و از گرسنگی UART و تداخل زمان‌بندی جلوگیری می‌کند.

۴.۱۲ فشرده‌سازی قاب مخابراتی

متغیرهای جداگانه بایت پایین و بالا حذف شدند. کمیت‌ها در ساختار ۱۶بیتی نگه داشته و هنگام ساخت قاب با تابع واحد سریال می‌شوند. آرایه ابتدا صفر است؛ بنابراین ناحیه‌های رزرو بدون ده‌ها انتساب صریح صفر باقی می‌مانند. حلقه دما جای ۶۴ انتساب تکراری را گرفته است. جایگاه هیچ فیلد سازگار تغییر نکرده است.

۵.۱۲ بازطراحی دریافت سریال

کد قبلی پس از اجرای هندلر HAL، ثبات داده را مستقیم می‌خواند و همان هندلر را دوباره صدا می‌زد. این ترتیب می‌توانست پرچم دریافت را پیش از خواندن پاک یا یک بایت را دوبار رسیدگی کند. نسخه جدید دریافت رسمی تک‌بایتی HAL را شروع می‌کند، callback فقط بافر را پر می‌کند و تفسیر دوازده‌بایت در حلقه اصلی انجام می‌شود. نوشتن Flash دیگر هرگز داخل ISR نیست.

۶.۱۲ بازطراحی حافظه شناسه‌ها

نسخه قدیمی از 0x08009000 می‌خواند، ولی هنگام فرمان تغییر، صفحه 0x08008000 را پاک می‌کرد. سپس ۶۴ مقدار هشت‌بایتی را از آرایه‌ای با ۳۲ عضو می‌نوشت. هر دو نشانی نزدیک انتهای کد ۳۲ کیلوبایتی قدیمی بودند و رشد برنامه می‌توانست داده یا کد را تخریب کند.

نسخه جدید آخرین صفحه واقعی قطعه را رزرو کرده است. لینک Keil و PlatformIO پیش از آن پایان می‌یابد. فقط ۳۲ شناسه نوشته می‌شوند و دو سطح CRC همراه با نسخه قالب وجود دارد. مهاجرت فقط جدول قدیمی کاملاً معتبر را می‌پذیرد. اگر برق میان پاک‌کردن و نوشتن قطع شود، رکورد نامعتبر تشخیص داده و پیش‌فرض استفاده می‌شود؛ نگهداری دو نسخه افزونه‌ای برای تحمل کامل قطع برق هنوز پیاده نشده است.

۷.۱۲ اصلاح رفتار کنترل

رابطه‌ها و ضرایب اصلی حفظ شدند، ولی تقسیم بر نقطه تنظیم صفر، رطوبت خارج از بازه و محاسبه پنجره بزرگ‌تر از صد مهار شدند. حالت خاموش خودکار دیگر در دور بعدی حلقه ناخواسته دوباره فعال نمی‌شود. فرمان دستی همچنان ممکن است در حالت خودکار توسط کنترل بعدی بازنویسی شود؛ این رفتار سازگار و در فصل پروتکل ثبت شده است.

۸.۱۲ مرزهای حفاظت شده CubeMX

در `main.c` فقط افزودن سرآیند کاربرد، ساخت ساختار دسته‌ها، فراخوانی راه‌اندازی و فراخوانی حلقه داخل نشانگرهای کاربر قرار دارد. فایل‌های `app.c`، `sensor\_address\_store.c` و درایورها خارج از مالکیت مولدند. با این حال، فایل پروژه Keil و اندازه حافظه ممکن است توسط مولد بازنویسی شوند و پس از هر تولید باید مطابق چک‌لیست فصل ساخت کنترل شوند.

۹.۱۲ مواردی که عمداً تغییر نکردند

نرخ سریال، طول فرمان، قاب ۲۰۹ بیتی، ترتیب کم‌بایت، شناسه‌های پیش‌فرض، نگاشت فازهای A و B، ضرایب کالیبراسیون، دو خروجی هم‌زمان سرمایش و منطق امکان کار هم‌زمان سرمایش و هیتر حفظ شده‌اند. بدون نرم‌افزار رایانه و شماتیک، تغییر این قراردادها ریسک ناسازگاری داشت.

۱۰.۱۲ موارد باقیمانده برای مرحله بعد

درایور یک‌سیمه باید به ماشین حالت غیرمسدودکننده با پالس زمان‌سنج، تبدیل هم‌زمان، تشخیص حضور و CRC حافظه موقت تبدیل شود. رابط انرژی باید نتیجه HAL و مهلت بازنشانی را برگرداند. پروتکل به همگام‌سازی، CRC، پاسخ و شماره نسخه نیاز دارد. حفاظت حداقل زمان کمپرسور، واچ‌داگ، ثبت خطا و آزمون واحد نیز هنوز انجام نشده‌اند.

فصل ۱۳

مرجع توابع تولیدشده و راهاندازی سختافزار

این فصل بخش‌هایی را شرح می‌دهد که عمدتاً توسط STM32CubeMX تولید می‌شوند. توضیحات برای فهم رفتارند؛ کامنت دستی خارج از ناحیه‌های کاربر نباید به این فایل‌ها افزوده شود.

۱.۱۳ فایل اصلی

۱.۱.۱۳ تابع main

نقطه ورود پس از کد راهانداز پردازنده است. ابتدا HAL، کلاک سامانه و به‌ترتیب پایه‌ها، DMA، SPI، UART، سه تایمر، I2C و ADC را راهاندازی می‌کند. سپس ساختار دسته‌های کاربرد ساخته و App_Init فراخوانی می‌شود. حلقه بی‌نهایت فقط App_Process را اجرا می‌کند. مقدار بازگشت نظری عدد صحیح است، ولی حلقه هرگز پایان نمی‌یابد.

۲.۱.۱۳ تابع SystemClock_Config

نوسان‌ساز خارجی هشت مگاهرتز، تقسیم ورودی یک و ضریب PLL برابر نه را انتخاب می‌کند. کلاک پردازنده و گذرگاه پرسرعت ۷۲ مگاهرتز، گذرگاه اول ۳۶ مگاهرتز و گذرگاه دوم ۷۲ مگاهرتز می‌شوند. کلاک ADC از تقسیم شش و برابر ۱۲ مگاهرتز است. هر خطای HAL به Error_Handler می‌رود.

۳.۱.۱۳ تابع MX_ADC1_Init

مبدل نخست را در حالت اسکن و تبدیل پیوسته با دو رتبه تنظیم می کند. کانال ۵ رتبه اول و کانال ۴ رتبه دوم اند و هر دو زمان نمونه برداری ۵.۲۳۹ سیکل دارند. هم ترازوی داده راست و شروع نرم افزاری است. DMA در تابع پشتیبانی پیرامونی به دسته متصل می شود.

۴.۱.۱۳ تابع MX_I2C1_Init

رابط نخست را با سرعت صد کیلوهرتز، آدرس دهی هفت بیتی، بدون آدرس دوم، فراخوان عمومی یا کشش غیر فعال راه اندازی می کند. پایه ها در تابع پشتیبانی به PB8 و PB9 بازنگاشت می شوند.

۵.۱.۱۳ تابع MX_SPI2_Init

رابط دوم را ارباب تمام دوطرفه، هشت بیتی، پرارزش ترین بیت ابتدا، قطبیت بالا و نمونه برداری لبه دوم می کند. انتخاب تراشه نرم افزاری و تقسیم کلاک ۶۴ است؛ با کلاک گذرگاه ۳۶ مگاهرتز نرخ تقریبی ۵.۵۶۲ کیلوبیت بر ثانیه حاصل می شود.

۶.۱.۱۳ توابع MX_TIM1_Init، MX_TIM2_Init و MX_TIM3_Init

هر سه تایمر کلاک داخلی و شمارش افزایشی دارند. پیش تقسیم گر ۹۹۹۹ کلاک ۷۲ مگاهرتز را به ۷۲۰۰ هرتز می رساند. دوره ۷۱۹۹ در تایمرهای یک و سه رویداد یک ثانیه ای و دوره ۳۵۹۹ در تایمر دو رویداد نیم ثانیه ای می سازد. پیش بارگذاری خودکار غیر فعال است.

۷.۱.۱۳ تابع MX_USART2_UART_Init

رابط دوم را با نرخ ۹۶۰۰، هشت بیت داده، یک توقف، بدون توازن و کنترل جریان، ارسال و دریافت هم زمان و نمونه برداری شانزده برابر تنظیم می کند.

۸.۱.۱۳ تابع MX_DMA_Init

کلاک کنترل کننده نخست را روشن و وقفه کانال یک را با اولویت پیش دستی دو و زیر اولویت یک فعال می کند. تنظیم جهت، اندازه داده و حالت چرخشی در تابع پشتیبانی ADC انجام می شود.

۹.۱.۱۳ تابع MX_GPIO_Init

کلاک درگاه‌های A تا E را روشن، سطح اولیه خروجی‌های قدرت و خطوط یک‌سیمه را ایمن و سپس حالت پایه‌ها را تنظیم می‌کند. خطوط قدرت پوش‌پول کم‌سرعت‌اند. درایور دما پس از این تابع دو پایه مربوط را به تخلیه‌باز تغییر می‌دهد. تنظیم اولیه بالا برای خروجی‌های فعال‌پایین مانع پالس روشن ناخواسته می‌شود.

۱۰.۱.۱۳ تابع Error_Handler

در نسخه مولد بدنه عملیاتی ندارد. بنابراین خطای راهاندازی فقط جریان همان تابع را برمی‌گرداند و سامانه ممکن است با پیرامونی ناقص ادامه دهد. تبدیل آن به خاموش‌سازی خروجی‌ها، ثبت علت و حلقه امن یا بازنشانی واچ‌داگ یک کار باز مهم است؛ پیاده‌سازی باید داخل ناحیه کاربر قرار گیرد.

۲.۱۳ توابع پشتیبانی پیرامونی

۱.۲.۱۳ تابع HAL_MspInit

کلاک رابط عملکرد جایگزین و توان را روشن می‌کند و رابط JTAG را غیرفعال ولی رابط SWD را فعال نگه می‌دارد. بنابراین پایه‌های برنامه‌ریزی دو سیمه در دسترس و برخی پایه‌های JTAG برای GPIO آزاد می‌شوند.

۲.۲.۱۳ توابع HAL_ADC_MspInit و HAL_ADC_MspDeInit

نسخه راهاندازی کلاک ADC و درگاه A را روشن، پایه‌های PA4/PA5 را آنالوگ، کانال نخست DMA را انتقال پیرامونی به حافظه با افزایش نشانی حافظه، داده نیم‌کلمه و حالت چرخشی می‌کند. دسته DMA به ADC پیوند و وقفه فعال می‌شود. نسخه توقف همه این منابع را آزاد می‌کند.

۳.۲.۱۳ توابع HAL_I2C_MspInit و HAL_I2C_MspDeInit

پایه‌های PB8/PB9 را خروجی عملکرد جایگزین تخلیه‌باز پرسرعت، بازنگاشت I2C را فعال و کلاک رابط را روشن می‌کنند. تابع توقف کلاک و پایه‌ها را آزاد می‌سازد.

۴.۲.۱۳ توابع HAL_SPI_MspInit و HAL_SPI_MspDeInit

برای رابط دوم، پایه کلاک و خروجی داده را عملکرد جایگزین پوش پول و ورودی داده را ورودی بدون کشش می کند. کلاک و وقفه SPI فعال می شوند. تابع توقف ترتیب معکوس را انجام می دهد.

۵.۲.۱۳ توابع پشتیبانی تایمرها

دو تابع HAL_TIM_Base_MspInit و HAL_TIM_Base_MspDeInit مسئول این بخش هستند. براساس نمونه تایمر، کلاک و وقفه تایمر یک، دو یا سه را فعال یا غیرفعال می کنند. اولویت تایمر سه از تایمر دو بالاتر و هر دو از تایمر یک بالاتر تنظیم شده است؛ پس از بازآرایی callbackها بسیار کوتاه اند.

۶.۲.۱۳ توابع HAL_UART_MspInit و HAL_UART_MspDeInit

کلاک رابط دوم و درگاه D را فعال، ارسال PD5 را عملکرد جایگزین و دریافت PD6 را ورودی می کند، بازنگاشت را فعال و وقفه UART را با اولویت بالا روشن می سازد. تابع توقف این تنظیمات را آزاد می کند.

۳.۱۳ هندلرهای استثنا و وقفه

توابع خطای سخت، مدیریت حافظه، خطای گذرگاه و دستور نامعتبر پس از ورود در حلقه بی نهایت می مانند. وقفه تیک سامانه شمارنده میلی ثانیه HAL را افزایش می دهد. وقفه های DMA، ADC، سه تایمر، SPI و UART فقط هندلر HAL متناظر را فراخوانی می کنند. هندلرهای سرویس سامانه، پایش اشکال و درخواست معوق بدنه کاربر ندارند. هیچ منطق محصولی نباید مستقیماً به این هندلرها بازگردانده شود.

۴.۱۳ فایل سامانه و راه انداز

فایل system_stm32f1xx.c متغیر کلاک هسته و تنظیمات پایه پیش از ورود به برنامه را فراهم می کند. فایل اسمبلی Keil جدول بردار، پشته، نقطه بازنشانی و انتقال به کتابخانه زمان اجرا را تعریف می کند. PlatformIO از نسخه متناظر چارچوب خود استفاده می کند و فایل سامانه محلی را برای جلوگیری از نماد تکراری کنار می گذارد.

فصل ۱۴

نشانگر زنده‌بودن و آزمون دو چراغ

۱.۱۴ هدف آزمون

دو چراغ متصل به پایه‌های PC13 و PC14 یک نشانگر مستقل و غیرمسدودکننده برای مشاهده اجرای برنامه فراهم می‌کنند. الگوی عادی فقط زمانی جلو می‌رود که تابع App_Process مرتب در حلقه اصلی فراخوانی شود. ثابت ماندن طولانی الگو، پس از در نظر گرفتن زمان طبیعی عملیات حسگرها، می‌تواند نشانه توقف حلقه اصلی یا گیرکردن یک تابع باشد. الگوی خطای سریع نیز ورود برنامه به Error_Handler را از اجرای عادی متمایز می‌کند.

این نشانگر جایگزین واچ‌داگ، ثبت خطا یا دیباگر نیست؛ چراغ فقط یک مشاهده سریع میدانی از زنده‌بودن حلقه اصلی ارائه می‌دهد.

۲.۱۴ اتصال الکتریکی و قطب فعال

هر چراغ باید مقاومت سری مستقل داشته باشد. برای کاهش بار پایه‌های دامنه پشتیبان درگاه سوم، مقاومت ۶۸۰ اهم تا یک کیلو اهم و چراغ کم‌جریان پیشنهاد می‌شود. اتصال پیش‌فرض نرم‌افزار فعال پایین است:

3.3 V -> series resistor -> LED anode -> LED cathode -> PC13 or PC14

در این اتصال، صفر منطقی چراغ را روشن می‌کند. اگر اتصال برد از خروجی پایه به مقاومت و سپس چراغ و زمین است، مقدار DIAGNOSTIC_LED_ACTIVE_LOW در فایل Core/Inc/diagnostic_led.h باید از یک به صفر تغییر کند. دو چراغ نباید یک مقاومت مشترک داشته باشند. زمین برد و پروگرامر نیز باید مشترک باشد.

۳.۱۴ تنظیمات مولد کد

در فایل `STM32F107.ioc` هر دو پایه به صورت خروجی پوش پول با سرعت پایین، بدون مقاومت کشش داخلی و سطح اولیه بالا ثبت شده اند. برچسب ها به ترتیب `DEBUG_LED_1` و `DEBUG_LED_2` هستند. منبع کلاک کم فرکانس خارجی فعال نیست؛ بنابراین استفاده از `PC14` به عنوان خروجی با تنظیم کلاک موجود تعارض ندارد.

ماژول عیب یابی هنگام راه اندازی همین تنظیم را دوباره و به صورت بی خطر اعمال می کند. این تکرار عمدی است تا اگر خطای راه اندازی پیش از پایان تابع تنظیم پایه ها رخ داد، امکان نمایش خطا باقی بماند. پس از هر تولید مجدد باید وجود دو پایه، برچسب ها، سطح اولیه بالا و سرعت پایین کنترل شود.

۴.۱۴ فایل ها و رابط عمومی

پیاده سازی در `Core/Src/diagnostic\_led.c` و رابط عمومی در `Core/Inc/diagnostic\_led.h` قرار دارد. فایل پیاده سازی مستقل از فایل های مولد است و به پروژه Keil نیز افزوده شده است؛ PlatformIO آن را به طور خودکار از پوشه سورس پیدا می کند.

۱.۴.۱۴ تابع `DiagnosticLed_Init`

ورودی و خروجی ندارد. کلاک درگاه GPIOC را فعال، دو پایه را خروجی پوش پول کم سرعت و هر دو چراغ را خاموش می کند. چند بار فراخوانی آن مجاز است، ولی هر بار حالت و گام الگو را بازنشانی می کند.

۲.۴.۱۴ تابع `DiagnosticLed_SetMode`

یک مقدار از نوع `DiagnosticLed_Mode` می گیرد، الگوی جدید را انتخاب و اجرای آن را از گام نخست آغاز می کند. مقدار خارج از محدوده به حالت خاموش تبدیل می شود. این تابع برای متن اصلی طراحی شده و نباید همزمان از وقفه تغییر داده شود.

۳.۴.۱۴ تابع `DiagnosticLed_Process`

ورودی و خروجی ندارد و باید با بیشترین تکرار ممکن در حلقه اصلی صدا زده شود. زمان را با `HAL_GetTick` می خواند و با تفریق بدون علامت، عبور شمارنده میلی ثانیه از بیشینه را نیز درست مدیریت می کند. اگر موعد گام بعد نرسیده باشد بلافاصله بازمی گردد؛ در نتیجه هیچ تأخیر مسدود کننده ای به سامانه تحمیل نمی شود.

اگر حلقه اصلی برای چند گام متوقف شود، تابع در اولین اجرای بعدی تعداد گام‌های گذشته را محاسبه و الگو را به موقعیت درست می‌برد. این رفتار از انباشته شدن تأخیر زمانی جلوگیری می‌کند.

۴.۴.۱۴ تابع DiagnosticLed_GetMode

ورودی ندارد و حالت فعال را برمی‌گرداند. این تابع برای مشاهده در دیباگر، گزارش وضعیت یا توسعه فرمان‌های عیب‌یابی آینده در نظر گرفته شده است و سخت‌افزار را تغییر نمی‌دهد.

۵.۱۴ الگوهای قابل انتخاب

۱.۵.۱۴ حالت DIAGNOSTIC_LED_MODE_OFF

هر دو چراغ خاموش می‌شوند. این حالت برای غیرفعال کردن آزمون بدون حذف کد است.

۲.۵.۱۴ حالت DIAGNOSTIC_LED_MODE_HEARTBEAT

دو تپ کوتاه و سپس مکث ایجاد می‌شود. این الگو نشانگر آرام برای بهره‌برداری دائمی است.

۳.۵.۱۴ حالت DIAGNOSTIC_LED_MODE_DANCE

چراغ اول، چراغ دوم، هر دو و سپس مکث اجرا می‌شود. چرخه تقریباً یک ثانیه است و حالت پیش‌فرض برنامه محسوب می‌شود.

۴.۵.۱۴ حالت DIAGNOSTIC_LED_MODE_ERROR

دو چراغ هر ۱۵۰ میلی‌ثانیه به سرعت جابه‌جا می‌شوند. این الگو نشان‌دهنده ورود به خطای بحرانی است.

۶.۱۴ اتصال به چرخه برنامه

تابع App_Init ماژول را راه‌اندازی و حالت رقص را انتخاب می‌کند. تابع App_Process پیش از رسیدگی به فرمان‌ها و نمونه‌برداری، یک بار تابع پردازش چراغ را اجرا می‌کند. اگر برنامه وارد Error_Handler شود، دو

پایه دوباره راهاندازی و حالت خطا در یک حلقه اختصاصی پردازش می‌شود.

برای استفاده آرام‌تر پس از پایان آزمون کافی است در Core/Src/app.c حالت اولیه از رقص به ضربان تغییر کند؛ حذف فراخوانی پردازش توصیه نمی‌شود، زیرا ارزش نشانگر زنده بودن از بین می‌رود.

۷.۱۴ پروگرام با رابط چهار سیمه

برای پروگرام معمولی اتصال‌های زمین مشترک، ولتاژ مرجع ۳.۳ ولت، داده رابط و کلاک رابط لازم‌اند. نام دو خط رابط به ترتیب SWDIO روی PA13 و SWCLK روی PA14 است. در صورت دشواری اتصال، خط بازنشانی نیز می‌تواند به عنوان سیم پنجم افزوده شود.

پس از اتصال J-Link و روشن کردن هدف، فرمان‌های زیر خروجی را ساخته و پروگرام می‌کنند:

```
pio run
pio run --target upload
```

در Keil نیز هدف STM32F107 باز، پروژه ساخته و گزینه بارگذاری انتخاب می‌شود. فایل MDK-ARM/ STM32F107.uvprojx از قبل مازول چراغ را در گروه کد کاربر دارد.

۸.۱۴ نتیجه پروگرام انجام شده

در تاریخ ۱۴۰۵/۰۶/۲۳، بارگذاری مستقیم از همین پروژه با PlatformIO و J-Link انجام شد. پروگرامر از طریق رابط USB شناسایی و ولتاژ مرجع هدف ۳۲۵.۳ ولت گزارش شد. قطعه STM32F107VC و هسته Cortex-M3 از مسیر SWD با موفقیت شناسایی شدند. پاک کردن، نوشتن، تطبیق محتوای حافظه و بازنشانی نهایی همگی بدون خطا پایان یافتند. بنابراین فایل اجرایی این نسخه روی سخت افزار بارگذاری شده است؛ مشاهده فیزیکی ترتیب چراغ‌ها باید توسط کاربر روی برد تأیید شود.

۹.۱۴ روند آزمون و تفسیر نتیجه

ابتدا برای جلوگیری از عملکرد ناخواسته، بارهای قدرت رله، هیتر و سرمایش قطع یا ایمن شوند. سپس برنامه بارگذاری و برد بازنشانی شود.

۱. در ثانیه نخست باید ترتیب چراغ اول، چراغ دوم و هر دو مشاهده شود.

۲. ادامه منظم چرخه نشان می دهد حلقه اصلی در حال اجرا و تیک سامانه فعال است.
 ۳. مکث یا پرش کوتاه می تواند ناشی از عملیات مسدودکننده فعلی حسگر دما یا ارسال سریال باشد و باید با دیباگر زمان سنجی شود.
 ۴. ثابت شدن طولانی الگو یعنی حلقه اصلی دیگر تابع پردازش را اجرا نمی کند یا تیک سامانه متوقف شده است.
 ۵. تناوب سریع و پیوسته چراغها یعنی برنامه وارد هندلر خطای بحرانی شده است.
 ۶. خاموش بودن دائمی هر دو چراغ می تواند از قطب اشتباه، سیم بندی نادرست، نبود مقاومت و تغذیه، پروگرام نشدن یا اجرانشدن برنامه ناشی شود.
- این آزمون فقط راه اندازی اولیه و زنده بودن نرم افزار را نشان می دهد. تأیید سنسورها، ارتباط سریال، حافظه و کنترل بارها باید مطابق چک لیست آزمون سامانه جداگانه انجام شود.

فصل ۱۵

راه‌اندازی ارتباط و اندازه‌گیری با ADE7880

۱.۱۵ دامنه این مرحله و نتیجه قابل انتظار

در این مرحله رابط SPI2 میکروکنترلر دوباره فعال، کتابخانه ADE7880 مستقل از خانواده میکروکنترلر بازنویسی و خواندن دوره‌ای کمیت‌های اصلی به برنامه افزوده شد. برنامه در هر نمونه، ولتاژ مؤثر، جریان مؤثر، توان اکتیو، توان ظاهری و ضریب توان هر سه فاز و همچنین جریان مؤثر نول را می‌خواند. داده خام کامل در متغیر `g_app_electrical` منتشر می‌شود و در پنجره Watch قابل مشاهده است. علاوه بر داده خام، ولتاژ تقریبی فازهای A/B با ضرایب بازیابی شده از برنامه قدیمی در آرایه `voltage_v` قرار می‌گیرد. معتبر بودن هر خانه و منبع ضریب آن به ترتیب در `voltage_valid` و `voltage_source` ثبت می‌شود.

خواندن رجیستر با تبدیل مقدار خام به واحد فیزیکی یکسان نیست. ضریب توان از قالب ثابت خود تراشه مستقیماً به بازه منفی یک تا مثبت یک تبدیل پذیر است، اما تبدیل دیگر کمیت‌ها به ولت، آمپر و وات وابسته به نسبت ترانس جریان، شبکه تقسیم ولتاژ، مقاومت بار، بهره ورودی و کالیبراسیون واقعی همان برد است. ضرایب قدیمی ولتاژ برای راه‌اندازی اولیه مفیدند، ولی تا آزمون با مرجع معتبر نباید مبنای حفاظت، ایمنی یا اندازه‌گیری نهایی قرار گیرند.

۲.۱۵ منبع بازیابی اتصال‌های قبلی

مخزن هنگام آغاز این مرحله هیچ تاریخچه Git نداشت، ولی فایل پشتیبان قدیمی همین پروژه در مسیر بیرونی `embedded/software/backup/Main/STM32F107.ioc` هنوز موجود بود. تنظیمات آن با گزارش قبلی و سورس قدیمی تطبیق داده شد. هر سه منبع اتصال زیر را تأیید کردند:

پایه میکرو	سیگنال	تنظیم لازم
PB12	ADE_CS / SS	خروجی پوش‌پول، فعال‌پایین، سطح بی‌کار بالا
PB13	SPI2_SCK	خروجی عملکرد جایگزین پوش‌پول
PB14	SPI2_MISO	ورودی بدون کشش داخلی
PB15	SPI2_MOSI	خروجی عملکرد جایگزین پوش‌پول

در فایل‌های قدیمی هیچ پایه‌ای برای RESET یا IRQ1 تراشه تعریف نشده بود. به همین دلیل نسخه حاضر از بازنشانی نرم‌افزاری و بررسی رجیستر STATUS1.RSTDONE استفاده می‌کند. اگر در شماتیک واقعی این دو خط به میکرو متصل‌اند، شماره پایه‌ها باید در مرحله بعد از روی شماتیک قطعی شود؛ تا آن زمان نباید پایه‌ای برای آن‌ها حدس زده شود.

۳.۱۵ کنترل الزامات دیتاشیت

مرجع این پیاده‌سازی دیتاشیت رسمی بازبینی C شرکت Analog Devices است: [ADE7880 Rev. C](#)

نکات زیر مستقیماً در طراحی رعایت شده‌اند:

- تراشه پس از روشن‌شدن تقریباً ۴۰ میلی‌ثانیه برای آماده‌شدن نیاز دارد؛ کتابخانه ۵۰ میلی‌ثانیه حاشیه می‌دهد.
- رابط پیش‌فرض پس از reset از نوع I2C است. سه گذار پایه SS از بالا به پایین رابط را به SPI تغییر می‌دهد.
- نوشتن رجیستر هشت‌بیتی CONFIG2 پس از انتخاب SPI، انتخاب رابط را قفل می‌کند.
- داده روی لبه بالارونده SCLK نمونه‌برداری و پرازش‌ترین بیت ابتدا منتقل می‌شود.
- بیشینه کلاک مجاز SPI برابر ۵.۲ مگاهرتز است. نرخ انتخاب‌شده ۵.۵۶۲ کیلوبیت‌برثانیه حاشیه کافی دارد.
- پایان reset نرم‌افزاری با دو شرط کنترل می‌شود: SWRST=0 و RSTDONE=1.
- پردازشگر داخلی تنها پس از نوشتن و بازخوانی موفق مقدار یک در رجیستر RUN شروع می‌شود.

۴.۱۵ تنظیم از صفر در STM32CubeMX

فایل STM32F107.ioc مرجع تنظیم مولد است. برای بازسازی دستی همین پیکربندی مراحل زیر انجام شود:

۱. فایل پروژه را در نسخه سازگار STM32CubeMX باز کنید و قطعه STM32F107VCTx با بسته LQFP100 را تغییر ندهید.

۲. از بخش Connectivity، رابط SPI2 را روی Full-Duplex Master قرار دهید.

۳. کنترل کنید PB13 برابر SPI2_SCK، PB14 برابر SPI2_MISO و PB15 برابر SPI2_MOSI شده باشد.

۴. پایه PB12 را GPIO Output با برچسب ADE_CS انتخاب کنید. سطح اولیه باید High، نوع خروجی Push-Pull، کشش No pull و سرعت High باشد.

۵. در تنظیم‌های SPI2 حالت ارباب، جهت دوخطه، طول هشت بیت، CPHA=2 Edge، CPOL=High، انتخاب تراشه نرم‌افزاری، MSB First و تقسیم‌کننده ۶۴ را انتخاب کنید.

۶. مقدار محاسبه‌شده کلاک باید با کلاک گذرگاه اول ۳۶ مگاهرتز برابر ۵.۵۶۲ کیلو بیت بر ثانیه باشد.

۷. فعال کردن وقفه SPI2 برای روش polling این مرحله لازم نیست. افزودن بی دلیل وقفه فقط پیچیدگی ایجاد می‌کند.

۸. در Project Manager > Code Generator گزینه نگهداری کد کاربر فعال و حذف فایل‌های قبلی غیرفعال باقی بماند.

۹. کد را تولید و سپس تفاوت‌ها را با Git بازبینی کنید. وجود MX_SPI2_Init و تنظیم پایه‌ها نتیجه مورد انتظار مولد است.

فراخوانی‌های برنامه داخل ناحیه‌های حفاظت‌شده فایل اصلی هستند:

```

۱ /* USER CODE BEGIN 2 */
۲ App_Init(&hspi2);
۳ /* USER CODE END 2 */
۴ /* USER CODE BEGIN WHILE */
۵ while (1) {
۶     App_Process();
۷ /* USER CODE END WHILE */
۸ }

```

در نتیجه، تولید مجدد CubeMX ممکن است بدنه‌های مولد MX_SPI2_Init و HAL_SPI_MspInit را بازسازی کند، اما منطق کتابخانه و برنامه در پوشه‌های Libraries/ADE7880 و Application پاک نمی‌شود و دو نقطه اتصال نیز داخل بلوک کاربر حفظ می‌شوند.

۵.۱۵ معماری کتابخانه قابل استفاده مجدد

کتابخانه فقط دو فایل دارد:

○ Libraries/ADE7880/Inc/ade7880.h: انواع، رجیسترها و رابط عمومی؛

○ Libraries/ADE7880/Src/ade7880.c: پروتکل، reset، خواندن و کالیبراسیون.

کتابخانه هدر stm32f1xx_hal.h را نیاز ندارد. وابستگی سخت‌افزار با ساختار ADE7880_Transport به آن تزریق می‌شود:

```

۱ typedef struct {
۲     void *context;
۳     ADE7880_Status (*write)(void *, const uint8_t *, size_t, uint32_t);
۴     ADE7880_Status (*read)(void *, uint8_t *, size_t, uint32_t);
۵     void (*select)(void *, bool active);
۶     void (*delay_ms)(void *, uint32_t);
۷ } ADE7880_Transport;

```

برای انتقال کتابخانه به میکرو دیگر فقط همین چهار callback پیاده می‌شود. برای مثال در STM32، callback نوشتن به HAL_SPI_Transmit و callback خواندن به HAL_SPI_Receive وصل شده است. در پروژه دیگر می‌توان آن‌ها را به درایور LL، ESP-IDF، Zephyr یا یک SPI نرم‌افزاری متصل کرد، بدون آن که فایل کتابخانه تغییر کند.

راهنمای مستقل و کوتاه استفاده از کتابخانه نیز در فایل Libraries/ADE7880/README.md قرار دارد؛ بنابراین برای انتقال ماژول به پروژه دیگر لازم نیست گزارش کامل محصول همراه آن کپی شود.

۶.۱۵ قرارداد کامنت‌گذاری و روش خواندن کد

تمام تعریف‌های عمومی کتابخانه در ade7880.h با قالب Doxygen انگلیسی مستند شده‌اند. هر تابع عمومی باید هدف، پارامترهای ورودی، خروجی و خطاهای قابل انتظار را داشته باشد. اعضای ساختار، مقدارهای enum

و رجیسترها نیز کنار تعریف خود توضیح انگلیسی دارند. فایل پیاده‌سازی علاوه بر توضیح هر تابع، دلیل مراحل غیر بدیهی مانند ترتیب بایت، بسط علامت، جلوگیری از انتشار نمونه ناقص و پاک کردن RSTDONE را ثبت می‌کند.

قاعده ادامه پروژه چنین است:

- نام تابع و متغیر باید هدف را بیان کند و کامنت جای نام‌گذاری نامناسب را نگیرد؛
- رابط عمومی با @brief، @param و @return انگلیسی مستند شود؛
- عدد ثابت سخت‌افزاری باید واحد و علت داشته باشد؛
- کامنت باید دلیل یا قرارداد را توضیح دهد، نه آن‌که همان دستور ساده را تکرار کند؛
- هر تغییر رفتار در گزارش فارسی و فایل وضعیت نیز ثبت شود.

۷.۱۵ مرجع سریع توابع کتابخانه

- ADE7880_Init: ثبت و کنترل چهار callback سخت‌افزار؛ هنوز با تراشه تبادل انجام نمی‌دهد.
 - ADE7880_Begin: انتخاب SPI، بازنشانی نرم‌افزاری، انتظار محدود و شروع DSP.
 - ADE7880_ReadRegister: خواندن عمومی رجیسترهای ۸، ۱۶ یا ۳۲ بیتی.
 - ADE7880_WriteRegisterVerified: نوشتن رجیستر و مقایسه بازخوانی برای تشخیص خطا.
 - ADE7880_ReadPhase: خواندن همه کمیت‌های خام یک فاز.
 - ADE7880_ReadAll: ساخت نمونه کامل سه فاز و نول؛ در خطا خروجی نیمه‌کاره منتشر نمی‌شود.
 - ADE7880_CalculateLinearCalibration: محاسبه scale/offset از دو نقطه خام و مرجع.
 - ADE7880_ApplyLinearCalibration: تبدیل یک شمار خام به واحد مهندسی.
 - ADE7880_Convert: تبدیل یک نمونه کامل با مجموعه ضرایب همه کانال‌ها.
 - App_SetVoltageCalibration: جایگزینی ضریب قدیمی یک فاز با ضریب اندازه‌گیری‌شده کاربر.
- ساده‌ترین مسیر استفاده در همین پروژه فقط دو فراخوانی دارد: یک بار App_Init(&hspi2) بعد از راه‌اندازی SPI2 و سپس اجرای پیوسته App_Process() در حلقه اصلی. لایه برنامه ساختار انتقال، بازیابی خطا، زمان نمونه‌برداری و تبدیل ولتاژ را مدیریت می‌کند؛ کاربر معمولاً فقط g_app_electrical را می‌خواند.

۸.۱۵ توالی راه اندازی برنامه

تابع App_Init پس از راه اندازی پایه ها و SPI2 فراخوانی می شود. توالی دقیق آن چنین است:

۱. ماژول دو LED راه اندازی می شود.
 ۲. callback های HAL در یک ADE7880_Transport قرار می گیرند.
 ۳. ADE7880_Init اعتبار callback ها را بررسی و CS را غیرفعال می کند.
 ۴. ADE7880_Begin پنجاه میلی ثانیه صبر، سه پالس CS ایجاد و SPI را با CONFIG2 قفل می کند.
 ۵. reset نرم افزاری با مهلت ۱۰۰ میلی ثانیه اجرا می شود؛ حلقه نامحدود وجود ندارد.
 ۶. رجیستر RUN نوشته و بازخوانی می شود.
 ۷. رجیستر هشتمین VERSION خوانده و در وضعیت دیباگ ثبت می شود.
 ۸. موفقیت حالت رقص LED و خطا تناوب سریع دو LED را فعال می کند.
- اگر ارتباط قطع باشد، برنامه در راه اندازی گیر نمی کند. وضعیت آفلاین ثبت و هر دو ثانیه تلاش مجدد انجام می شود. پس از بازیابی ارتباط، الگوی LED خودکار به رقص برمی گردد. در حالت سالم یک snapshot کامل تقریباً هر یک ثانیه خوانده می شود.

۹.۱۵ رجیسترها و قالب داده خوانده شده

کمیت	فاز A	فازهای B/C	تفسیر
جریان مؤثر	0x43C0	0x43C2/0x43C4	مقدار ۲۴ بیتی بدون علامت کاربردی
ولتاژ مؤثر	0x43C1	0x43C3/0x43C5	مقدار ۲۴ بیتی بدون علامت کاربردی
توان اکتیو	0xE513	0xE514/0xE515	مقدار ۲۴ بیتی علامت دار با بسط علامت
توان ظاهری	0xE519	0xE51A/0xE51B	داده ۲۴ بیتی در کلمه ۳۲ بیتی
ضریب توان	0xE902	0xE903/0xE904	عدد علامت دار Q1.15؛ هر LSB برابر 2^{-15}

کمیت	فاز A	فازهای B/C	تفسیر
جریان نول	0x43C6	ندارد	مقدار ۲۴ بیتی

در نسخه قدیمی ضریب توان از رجیستر هارمونیک مشترک FPF خوانده می‌شد و انتخاب فاز نیز درست تضمین نشده بود. نسخه حاضر مستقیماً از سه رجیستر APF/BPF/CPF استفاده می‌کند؛ بنابراین هم ساده‌تر و هم مطابق بخش Power Factor دیتاشیت است.

۱۰.۱۵ خواندن در VS Code و PlatformIO

پس از بازکردن ریشه پروژه، افزونه PlatformIO IDE را نصب و محیط stm32f107vc_jlink را انتخاب کنید. ساخت از ترمینال:

```
pio run -e stm32f107vc_jlink
```

اگر فرمان pio در PATH نیست، روی ویندوز می‌توان فایل اجرایی محیط خود PlatformIO را اجرا کرد:

```
%USERPROFILE%\platformio\penv\Scripts\platformio.exe run \
-e stm32f107vc_jlink
```

برای پروگرام:

```
pio run -e stm32f107vc_jlink --target upload
```

برای دیباگ، breakpoint را بعد از انتساب g_app_electrical.raw در App_Process قرار دهید و متغیر g_app_electrical را به Watch اضافه کنید. نتیجه سالم باید چنین ویژگی‌هایی داشته باشد:

◦ online=true

◦ last_status=ADE7880_STATUS_OK

◦ die_version دارای مقدار ثابت و تکرارپذیر؛

◦ افزایش successful_samples تقریباً هر ثانیه؛

◦ تازه‌شدن updated_at_ms

◦ ثابت‌ماندن communication_errors در صفر.

۱۱.۱۵ ساخت و دیباگ در Keil MDK

فایل MDK-ARM/STM32F107.uvprojx باز شود. کتابخانه از قبل در گروه Libraries/Reusable عضو است و مسیر include آن نیز ثبت شده است. بعد از تولید مجدد CubeMX موارد زیر کنترل شود:

۱. فایل‌های app.c و diagnostic_led.c از پوشه Application/Src هنوز عضو پروژه باشند.

۲. فایل Libraries/ADE7880/Src/ade7880.c فقط یک بار عضو پروژه باشد.

۳. مسیرهای Application/Inc و Libraries/ADE7880/Inc در Include Paths باشند.

۴. نمادهای USE_HAL_DRIVER و STM32F107xC تعریف شده باشند.

۵. هدف با گزینه Build Target بدون warning ساخته شود.

در پنجره Watch همان متغیر g_app_electrical اضافه شود. برای جلوگیری از توقف طولانی سیستم قدرت، breakpoint تنها هنگام جدا بودن بارها استفاده شود. اگر برنامه باید آزاد اجرا شود، از Watch با به‌روزرسانی دوره‌ای و بدون توقف مکرر بهره بگیرید.

ساخت واقعی همین پروژه با Keil و Arm Compiler 6.24 اجرا شد و با نتیجه 0 Error(s), 0 Warning(s) پایان یافت. اندازه‌های گزارش‌شده برابر Code=9628، RO-data=476، RW-data=12 و ZI-data=1460 بایت بودند و فایل HEX نیز تولید شد.

خطاهای اولیه همگی از ناشناخته‌بودن DMA_HandleTypeDef آغاز می‌شدند. فایل پیاده‌سازی DMA مربوط به HAL در پروژه وجود داشت، ولی تعریف HAL_DMA_MODULE_ENABLED در Core/Inc/stm32f1xx_hal_conf.h غیرفعال بود. با فعال کردن این تعریف، نوع‌های مورد نیاز handleهای I2C/SPI/TIM/UART پیش از استفاده در هدرهای HAL تعریف شدند. این یک وابستگی تعریفی در HAL است و به‌تنهایی به معنی استفاده منطق برنامه از انتقال DMA نیست.

۱۲.۱۵ کالیبراسیون دو نقطه‌ای

نوع ADE7880_LinearCalibration دو عدد scale و offset دارد و رابطه زیر را اعمال می‌کند:

$$\text{engineering} = \text{raw} * \text{scale} + \text{offset}$$

برای نمونه، فرض کنید در دو نقطه مرجع ۱۰۰ و ۲۳۰ ولت، رجیستر ولتاژ به‌ترتیب ۱۱۰۰۰۰۰ و ۲۵۵۰۰۰۰ باشد. محاسبه ضریب بدون دست‌کاری دستی فرمول چنین است:

```

۱ ADE7880_LinearCalibration voltage_a;
۲
۳ if (ADE7880_CalculateLinearCalibration(
۴     1100000.0F, 100.0F,
۵     2550000.0F, 230.0F,
۶     &voltage_a) == ADE7880_STATUS_OK) {
۷     App_SetVoltageCalibration(ADE7880_PHASE_A, &voltage_a);
۸ }

```

سپس ADE7880_Convert یک snapshot خام و ساختار ضرایب را به مقادیر مهندسی تبدیل می‌کند. بهتر است ضرایب هر فاز مستقل باشند. روند عملی کالیبراسیون:

۱. برد و مرجع اندازه‌گیری حداقل تا رسیدن دما و تغذیه به حالت پایدار روشن بمانند.
۲. برای هر فاز دو یا چند نقطه دور از هم انتخاب شود؛ نقطه صفر به‌تنهایی برای RMS مناسب نیست.
۳. در هر نقطه میانگین چند ده snapshot خام ثبت شود.
۴. مقدار مرجع کالیبره‌شده هم‌زمان ثبت و scale/offset محاسبه شود.
۵. ضرایب با شماره سریال برد، تاریخ، ابزار مرجع و شرایط آزمون ذخیره شوند.
۶. پس از اعمال ضرایب، نقطه سوم که در محاسبه استفاده نشده کنترل شود.

۱.۱۲.۱۵ ضرایب ولتاژ بازیابی‌شده و خروجی فعلی

در سورس قدیمی همین برد تبدیل زیر پیدا شد:

$$\text{phase A volts} = \text{AVRMS} * 0.00055963 * 1.017$$

$$\text{phase B volts} = \text{BVRMS} * 0.00055963$$

این دو ضریب اکنون هنگام App_Init بارگذاری می‌شوند. پس از نخستین نمونه موفق، خروجی‌های زیر در Watch قابل استفاده‌اند:

○ voltage_v[0]: ولتاژ تقریبی فاز A؛

○ voltage_v[1]: ولتاژ تقریبی فاز B؛

- `voltage_valid[0/1]=true`: تبدیل انجام شده است؛
 - `voltage_source[0/1]=APP_VOLTAGE_CALIBRATION_LEGACY`: ضریب از برنامه قدیمی آمده است؛
 - `voltage_valid[2]=false`: برای فاز C ضریب قدیمی معتبر پیدا نشده است.
- با فراخوانی `App_SetVoltageCalibration` منبع ضریب همان فاز «کاربر» ثبت می‌شود و نمونه بعدی با ضریب جدید منتشر خواهد شد. برای تکمیل کالیبراسیون جریان و توان هنوز نسبت CT، مقدار مقاومت `burden`، شبکه ورودی و محدوده نامی لازم است.

۲.۱۲.۱۵ مشاهده جریان، توان و ضریب توان در دیباگر

دراپور از ابتدا رجیسترهای جریان RMS، توان اکتیو، توان ظاهری و ضریب توان را نیز در هر `snapshot` می‌خواند. این داده‌ها در ساختار خام زیر قرار دارند:

- `raw.phase[0]` برای فاز A؛
- `raw.phase[1]` برای فاز B؛
- `raw.phase[2]` برای فاز C؛
- `raw.neutral_current_rms` برای جریان نول.

برای نمونه، پیش از انجام هر کالیبراسیون می‌توان در `Watch` ساختار `g_app_electrical.raw.phase[0]` را باز کرد و اعضای `current_rms`، `active_power`، `apparent_power` و `power_factor_q15` را برای فاز A دید. این مقادیر به‌جز ضریب توان، شمار خام پردازشگر داخلی تراشه هستند و واحد آمپر یا وات ندارند. نسخه جدید برنامه فیلدهای مستقیم و خوانای زیر را نیز منتشر می‌کند:

فیلد در <code>g_app_electrical</code>	معنی و شرط استفاده
<code>current_a</code>	آرایه جریان RMS برحسب آمپر.
<code>active_power_w</code>	آرایه توان اکتیو برحسب وات.
<code>apparent_power_va</code>	آرایه توان ظاهری برحسب ولت‌آمپر.
<code>power_factor</code>	آرایه ضریب توان علامت‌دار در بازه -۱ تا +۱.

معنی و شرط استفاده	فیلد در g_app_electrical
جریان نول برحسب آمپر؛ فقط با پرچم اعتبار متناظر قابل استفاده است.	neutral_current_a

سه عضو هر آرایه به ترتیب فازهای A/B/C هستند. برای جریان، توان اکتیو و توان ظاهری به ترتیب پرچم‌های زیر کنترل شوند:

○ current_valid

○ active_power_valid

○ apparent_power_valid

ضریب توان پس از یک snapshot کامل با power_factor_valid=true قابل استفاده است.

رجیسترهای APF/BPF/CPF اعداد علامت‌دار Q1.15 هستند؛ بنابراین هر LSB وزن 2^{-15} دارد و برنامه آن‌ها را با تقسیم بر ۳۲۷۶۸ به ضریب توان تبدیل می‌کند. این تبدیل به نسبت CT یا شبکه ولتاژ وابسته نیست. علامت PF مطابق قرارداد تراشه اطلاعات پیش‌فاز/پس‌فاز بودن را نیز حمل می‌کند.

در مقابل، مقدار آمپر و وات به سخت‌افزار همان برد وابسته است. به همین دلیل برنامه تا زمان نصب ضریب واقعی، فیلدهای مهندسی جریان و توان را صفر و پرچم اعتبارشان را نادرست نگه می‌دارد. این رفتار مانع اشتباه‌گرفتن شمار خام یا ضریب حدسی با اندازه‌گیری واقعی می‌شود.

۳.۱۲.۱۵ کالیبراسیون جریان و توان برای خروجی Watch

در دو نقطه پایدار و جدا از هم، شمار خام و مقدار هم‌زمان ابزار مرجع ثبت شود. برای جریان فاز A نمونه کد زیر قابل استفاده است:

```

۱ ADE7880_LinearCalibration phase_a_current;
۲
۳ if (ADE7880_CalculateLinearCalibration(
۴     raw_at_1_a, 1.0F,
۵     raw_at_5_a, 5.0F,
۶     &phase_a_current) == ADE7880_STATUS_OK) {
۷     (void)App_SetPhaseCalibration(
۸         APP_PHASE_QUANTITY_CURRENT_RMS,
۹         ADE7880_PHASE_A,
۱۰        &phase_a_current);
۱۱ }

```

برای توان اکتیو و ظاهری همین الگو به ترتیب با ثابت‌های زیر تکرار می‌شود:

```

۱ APP_PHASE_QUANTITY_ACTIVE_POWER
۲ APP_PHASE_QUANTITY_APPARENT_POWER

```

جریان نول با App_SetNeutralCurrentCalibration تنظیم می‌شود. ضریب‌ها فعلاً در RAM هستند و پس از reset از بین می‌روند؛ پس از تأیید نقطه سوم باید در آغاز برنامه یا حافظه غیرفرار بارگذاری شوند.

توان نوشته‌شده روی بدنه لامپ فقط مقدار نامی است و جای وات‌متر یا منبع کالیبره را نمی‌گیرد. برای نتیجه قابل اعتماد، ولتاژ، جریان، وات و PF مرجع باید هم‌زمان با داده خام ثبت شوند. راهنمای رسمی AN-1171 نیز کالیبراسیون gain را برای هر دستگاه لازم می‌داند و برای RMS استفاده از رجیسترهای لحظه‌ای و ثابت‌های A/LSB و V/LSB اندازه‌گیری‌شده را توصیه می‌کند.

۱۳.۱۵ آزمون سخت‌افزاری مرحله به مرحله

به علت اتصال ورودی‌های ADE7880 به برق شهر، این آزمون فقط با ایزولاسیون مناسب، منبع محدودشده و فرد آگاه به ایمنی برق انجام شود. اتصال مستقیم اسیلوسکوپ یا دیباگر زمین‌شده به بخش غیرایزوله می‌تواند خطرناک و مخرب باشد.

۱.۱۳.۱۵ آزمون بدون اعمال ولتاژ اندازه‌گیری

۱. بارهای رله و ورودی‌های قدرت را ایمن یا جدا کنید.

۲. اتصال SWCLK=PA14، SWDIO=PA13، زمین مشترک و مرجع ۳.۳ ولت J-Link را برقرار کنید.
۳. برنامه را بسازید، پروگرام و reset کنید.
۴. اگر ADE7880 پاسخ دهد، LEDها باید الگوی رقص تقریباً یک‌ثانیه‌ای داشته باشند.
۵. اگر ارتباط برقرار نشود، دو LED باید سریع یکی‌درمیان شوند و communication_errors پس از تلاش‌های خواندن افزایش یابد.
۶. با logic analyzer کنترل کنید CS در هر تراکنش پایین، SCLK در بی‌کاری بالا و نرخ آن تقریباً ۵.۵۶۲ کیلوهرتز باشد.
۷. فرمان خواندن باید با 0x01 و فرمان نوشتن با 0x00 آغاز شود و سپس نشانی ۱۶ بیتی MSB-first بیاید.

۲.۱۳.۱۵ آزمون با منبع مرجع

۱. ابتدا ولتاژ کم و ایمن مطابق طراحی ورودی اعمال کنید و تغییر رجیستر متناظر را ببینید.
۲. فقط پس از تأیید سیم‌بندی، منبع AC مرجع و بار کنترل‌شده وصل شود.
۳. فاز A و سپس B جداگانه تغییر داده شوند تا نگاشت کانال‌ها قطعی شود.
۴. علامت توان اکتیو با جهت توان کنترل شود؛ برعکس‌بودن CT می‌تواند علامت را معکوس کند.
۵. ضریب توان یک بار مقاومتی باید به مثبت یک نزدیک باشد. بار القایی/خازنی برای کنترل علامت و رفتار PF استفاده شود.
۶. قطع موقت سیم SPI یا تغذیه ADE باید حالت خطا ایجاد کند و پس از رفع عیب، تلاش دوره‌ای باید ارتباط را بازیابی کند.
۷. آزمون پیوسته حداقل ۲۴ ساعت برای شمار خطا، پایداری snapshot و الگوی LED انجام شود؛ آزمون چندروزه و چرخه دما برای ادعای صنعتی لازم است.

۱۴.۱۵ راهنمای عیب‌یابی

نشانه	بررسی پیشنهادی
LED خطا از ابتدای روشن شدن	تغذیه و کلاک کریستال ADE، اتصال PM0/PM1، سطح RESET، چهار خط SPI، سطح CS و مشترک بودن زمین بررسی شود.
تمام خواندن‌ها 0xFFFFFFFF	مسیر MISO باز، CS غیرفعال، تراشه بی‌تغذیه یا رابط هنوز روی I2C است.
تمام خواندن‌ها صفر	کلاک و ورودی‌ها، RUN، اتصال کوتاه MISO یا reset دائمی بررسی شود.
خطای متناوب	طول سیم، نویز، زمین، تغذیه، خازن‌های بای‌پس و شکل موج SCLK/CS با logic analyzer بررسی شود.
PF درست ولی ولت/آمپر نادرست	ابتدا داده raw را کنترل و سپس نسبت ورودی و ضرایب کالیبراسیون را اصلاح کنید.
ناشناخته بودن نوع DMA در Keil	فعال بودن ماژول DMA در تنظیمات HAL و عضویت سورس آن در پروژه بررسی شود.
پس از CubeMX کد کامپایل نمی‌شود	عضویت فایل‌های برنامه و کتابخانه، مسیرهای include و تعریف ADE_CS بررسی شود.

در همین خطا وجود فراخوانی App_Init(&hspi2) در بلوک کاربر فایل اصلی نیز کنترل شود.

۱۵.۱۵ نتیجه اعتبارسنجی نرم‌افزاری این مرحله

ساخت تازه با PlatformIO ststm32 20.0.0 و framework-stm32cubef1 1.8.7 با گزینه‌های -Wall -Werror -Wextra موفق شد. خروجی ۹۶۴۰ بایت Flash و ۴۷۲ بایت RAM مصرف کرد. ساخت دوم با Keil و Arm Compiler 6.24 نیز با 0 Error(s), 0 Warning(s) پایان یافت و اندازه‌های Code=9628, RO-data=476, RW-data=12 و ZI-data=1460 بایت را گزارش کرد. فایل‌های اجرایی هر دو زنجیره‌ابزار تولید شدند. کاربر برقراری ارتباط، دیباگ Keil و مشاهده ولتاژ یک بار لامپی روی برد واقعی را گزارش کرد؛ صحت جریان و توان مهندسی هنوز به ثبت داده مرجع و کالیبراسیون نیاز دارد.

۱۶.۱۵ چک‌لیست پذیرش این قابلیت

این مرحله زمانی از نظر سخت‌افزاری پذیرفته می‌شود که همه موارد زیر ثبت شوند:

- build بدون warning در Keil و PlatformIO؛
- تولید مجدد CubeMX و باقی‌ماندن منطق Application/Library؛
- مشاهده کلاک، CS و قالب فرمان صحیح با logic analyzer؛
- online=true و افزایش پیوسته شمار نمونه؛
- عدم افزایش خطای ارتباط در آزمون طولانی؛
- صحت نگاشت فازهای A، B و C و کانال نول؛
- کالیبراسیون هر کانال با مرجع معتبر و ثبت ضرایب؛
- بازیابی خودکار پس از قطع و وصل کنترل‌شده ارتباط یا تغذیه؛
- ثبت نتیجه، تاریخ، نسخه commit و تجهیزات آزمون در پرونده پروژه.